

Travail de Bachelor

Auto-battler multijoueur

Étudiant	Nicolas Duprat
Superviseur	Prof. Donini Pier
Département	Technologies de l'information et de la communication (TIC)
Filière	Informatique et systèmes de communication (ISC)
Orientation	Informatique Logicielle (ISCL)
Travail proposé par	Nicolas Duprat
Année académique	2026-27

Yverdon-les-Bains, le 24.07.2026

Authentification

Par la présente, j'atteste avoir réalisé ce travail et n'avoir utilisé aucune autre source que celles expressément mentionnées.

A handwritten signature in black ink, consisting of several loops and a long, sweeping underline.

Nicolas Duprat

Yverdon-les-Bains, le 24.07.2026

Préambule

Ce travail de Bachelor (ci-après TB) est réalisé en fin de cursus d'études, en vue de l'obtention du titre de Bachelor of Science HES-SO en Ingénierie.

En tant que travail académique, son contenu, sans préjuger de sa valeur, n'engage ni la responsabilité de l'auteur, ni celles du jury du travail de Bachelor et de l'Ecole.

Toute utilisation, même partielle, de ce TB doit être faite dans le respect du droit d'auteur.

HEIG-VD

Le Chef de département TIC

Yverdon-les-Bains, le 24.07.2026

Table des matières

Authentification	2
Préambule	3
Cahier des charges	7
Description du projet	7
Technologies utilisées	7
Valeur d'ingénierie du projet	7
Fonctionnalités attendues	8
Durée du projet	9
Livrables	9
Jalon	9
Présentation du jeu	10
Présentation d'un auto-battler	10
Mécanique principale de notre jeu	10
Déroulement de la partie	12
1 Introduction	13
1.1 Contexte motivation	13
1.2 Description du sujet	13
1.3 Architecture du projet	14
1.4 Planification	15
2 State of the art	16
2.1 Introduction	16
2.2 Frontend	16
2.3 Authentification	18
2.4 Connexion client-serveur	19

2.5	Gestion des connexions	19
2.6	Lien entre la base de données et le backend	20
2.7	CI/CD	21
3	Architecture logicielle	22
3.1	Schéma de l'architecture	23
3.2	Frontend	24
3.3	Backend	24
3.4	Base de données	25
3.5	Pipeline CI/CD	26
3.6	GitHub Actions	26
3.7	Docker	26
4	Conception	28
4.1	Conception Orientée Objet	28
4.2	Modèle des données	34
4.3	Architecture réseau	44
4.4	Système de simulation des combats	49
5	Implémentation	52
5.1	Configuration du projet	52
5.2	Authentification	53
5.3	Communication réseau	54
5.4	Base de données	58
5.5	Algorithmes	60
6	Difficultés rencontrées	68
6.1	Première base de données	68
6.2	Découverte de Play et notamment Pekko avec le pattern acteur	68
6.3	difficulté de focus sur les technologie utile	69
6.4	Difficulté de conception et de poser les choses à plat	69
6.5	Confusion entre vision du jeu et besoins logiciels	70
7	Conclusion	71
7.1	Bilan du projet	71
7.2	Limitations et perspectives	72
7.3	Apprentissages personnels	72
8	Journal de travail	74

Bibliographie	77
Annexes	81
I Outils utilisés	82

Cahier des charges

Description du projet

Le projet a pour but de créer un jeu ayant un style similaire à des jeux comme Teamfight Tactics ou Hearthstone battleground mais en le personnalisant. L'objectif n'est pas de réinventer le mode de jeu, mais de créer un jeu fonctionnel avec lequel les joueurs s'amuseront avec des personnages de la pop-culture. On cherche donc à réaliser un jeu multijoueur en ligne de type auto-battler, permettant à huit joueurs de s'affronter simultanément au cours d'une même partie.

Technologies utilisées

Pour ce projet, j'ai utilisé le framework Play pour le développer. Il a pour objectif d'être jouable sur navigateur internet. Pour le stockage des données, le projet utilise le système de gestion de base de données PostgreSQL.

Valeur d'ingénierie du projet

L'intérêt du projet réside dans la réalisation d'un jeu multijoueur entièrement en ligne, accessible directement depuis un navigateur web et ne nécessitant aucune installation de la part des utilisateurs. Cette approche permet de réduire les contraintes techniques à l'entrée et d'offrir une expérience de jeu immédiatement accessible, favorisant ainsi l'engagement et la participation des joueurs.

L'aspect multijoueur introduit des défis techniques importants, notamment la synchronisation d'état entre plusieurs clients, la gestion de la latence réseau, la cohérence des actions concurrentes et la fiabilité des communications. Le serveur doit également être capable d'orchestrer plusieurs parties simultanées tout en garantissant l'intégrité des données et la continuité de l'expérience de jeu. Pour répondre à ces exigences, il faudra que le design de la base de données permette une résistance à la panne, afin de limiter les risques de perte de données et de garantir la disponibilité du service.

La conception de la base de données doit aussi permettre un système de "replay" de partie afin que l'on puisse rejouer entièrement des parties qui ont été enregistrées sur la base de données. Cette fonctionnalité offrira non seulement aux joueurs la possibilité de revoir leurs parties, mais constituera également un outil précieux pour l'analyse de parties, le débogage et l'amélioration continue du jeu.

Fonctionnalités attendues

Cette section décrit les fonctionnalités attendues à la fin du projet. Elle contient également des fonctionnalités futures souhaitées.

Fonctionnalités obligatoires

- Un système d'authentification
- Un système de stockage des données utilisateurs
- Une page où chaque utilisateur peut regarder ses statistiques, soit le nombre de victoires dans un premier temps
- Un système de connexion multijoueur, où l'on pourra connecter 8 personnes à la même partie afin qu'ils s'affrontent en temps réel
- Un système de partie opérationnel : un joueur pourrait lancer une partie pour jouer contre d'autres joueurs ou des bots. Donc cela inclut :
 - Une boutique pour acheter les unités
 - Un système de combat qui fera s'affronter les unités de deux joueurs à intervalles réguliers
 - Un système de gestion de son équipe afin que l'on puisse changer soit la formation de son équipe ou bien les unités constituant mon équipe.
- Un système de replay de partie

Fonctionnalités optionnelles

- Un système de classe et de famille pour les unités, leur attribuant des effets spécifiques
- Une liste de 40 unités différentes jouables

Durée du projet

Une charge de 13 hebdomadaire devrait être investie le long du semestre ainsi que 45h supplémentaires à temps plein. Cela fait un total de 450 h de travail disponibles.

Livrables

- Remise du cahier des charges pour le 27.02.2026
- Remise du rapport intermédiaire le 20.05.2026 avant 15 h
- Remise du travail de Bachelor, du rapport, d'une affiche et un résumé publiable pour GAPS le 24.07.2026 avant 11 h.

Jalon

Étape 1 : Planification et définition du projet, rédaction du cahier des charges et du planning

Étape 2 : Analyse des besoins ainsi que leur spécification Étape 3 : Conception et réalisation de l'architecture (stack, et UML de classe), conception de la base de donnée et conception de l'interface utilisateur

Étape 4 : Implémentation des Fonctionnalités attendues Étape 5 : Implémentation des Fonctionnalités optionnelles Étape 6 : Réalisation des documents à rendre (affiche et résumé publiable)

Présentation du jeu

Maintenant que nous disposons d'une meilleure vue d'ensemble du projet, nous allons rappeler les principes fondamentaux d'un auto-battler avant de présenter les règles du jeu.

Présentation d'un auto-battler

Un auto-battler est un genre de jeu dans lequel chaque joueur constitue progressivement une équipe de personnages afin d'affronter les autres joueurs. Les combats se déroulent automatiquement, sans intervention du joueur, d'où le terme *auto-battler*. Le joueur doit donc adapter et renforcer son équipes pour maximiser ses chances de victoire.

À l'issue de chaque combat, le perdant perd une partie de ses points de vie. La partie se termine lorsqu'il ne reste plus qu'un seul joueur en vie.

Voyons maintenant plus en détail les mécaniques du jeu.

Mécanique principale de notre jeu

Chaque joueur dispose d'un terrain composé d'hexagones et d'un banc. Le terrain est séparé en 2 partie, une première partie où l'utilisateur va pouvoir placer son équipe, et une deuxième partie qui servira à accueillir l'équipe adverse lors des combats. Le banc est un espace de stockages des unités qui ne sont utilisés pour l'équipe actuelles.

Chaque joueur dispose d'une réserve d'or ainsi que l'accès à un magasin lui permettant d'acheter des unités parmi les cinq unités actuellement proposées. Les unités achetées sont alors immédiatement placées sur son banc.

Les joueurs possèdent également un niveau qui détermine le nombre maximal d'unités pouvant être déployées sur le terrain. Ils peuvent dépenser de l'or pour gagner de l'expérience, acheter des unités ou changer les unités présentent dans le magasin.

Enfin, chaque joueur possède un inventaire contenant des objets pouvant être équipés sur les unités afin d'améliorer leurs statistiques. Chaque joueur commencerait la partie avec trois objets aléatoires et en gagnerait trois autres après cinq combats.

Les unités sont classées selon 5 niveaux de rareté : Commune, Peu commune, Rare, Épique et Légendaire. Plus une unité est rare, plus son coût est élevé et plus le joueur doit avoir d'un niveau élevé pour avoir des chances de la voir apparaître dans le magasin.

Les unités possèdent également un niveau, où chaque passage au niveau supérieur les rend plus puissantes. Lorsqu'un joueur possède trois exemplaires d'une même unité, celles-ci fusionnent automatiquement pour former une unité de niveau supérieur.

Chaque unité a ses propres statistiques, qui déterminent son efficacité au combat.

- **Points de vie** : quantité de dégâts que l'unité peut subir avant de mourir.
- **Mana initial** : quantité de mana avec laquelle l'unité commence le combat
- **Mana maximal** : la quantité de mana nécessaire pour lancer sa compétence
- **Attaque de base** : les dégâts que l'unité inflige avant application de multiplicateur ou de mitigation des dégâts
- **Dégâts d'attaque** : un multiplicateur appliqué à l'attaque de base pour déterminer les dégâts totaux
- **Puissance** : utilisé principalement pour déterminer l'efficacité de la compétence
- **Vitesse d'attaque** : fréquence des attaques
- **Armure** : mitige les dégâts d'attaque
- **Résistance magique** : mitige les dégâts de compétences
- **Portée** : distance maximale d'attaque

Enfin, au-delà de leurs statistiques, chaque unité dispose également d'une capacité unique qui leur confère un rôle particulier sur le champ de bataille.

Au cours d'un combat, les unités gagnent du mana à chaque attaque effectuée. Lorsque leur réserve de mana atteint sa valeur maximale, elles lancent automatiquement leur compétence. Selon l'unité concernée, cette capacité peut infliger des dégâts, renforcer des alliés, affaiblir des ennemis ou produire des effets spéciaux modifiant le déroulement du combat.

Les objets permettent d'améliorer les statistiques des unités qui les équipent. Une même unité peut porter jusqu'à trois objets simultanément.

L'objectif du joueur est donc de gérer son économie, améliorer ses unités, composer une équipe efficace et survivre jusqu'à être le dernier joueur encore en vie.

Déroulement de la partie

Une partie oppose 8 joueurs. Au début de la partie, chaque joueur :

- Commence au niveau 1
- Possède une unité commune aléatoire de niveau 1
- Dispose de 30 pièces d'or

La partie est ensuite découpée en une succession de manches. Chaque manche comporte deux phases :

Phase de préparation:

Pendant 90 secondes, chaque joueur peut:

- Déplacer une unité du banc à son équipe ou inversement.
- Acheter une unité présente dans le magasin. Son coût varie entre 1 et 5 pièces d'or selon sa rareté.
- Vendre une unité qui est dans son équipe ou sur son banc, afin de récupérer son coût d'achat ainsi que les objets dont elle était équipée.
- Équiper un objet sur une unité
- Rafraîchir son magasin pour 2 pièces d'or
- Acheter 4 point d'expérience pour 4 pièces d'or.

Phase de combat : Chaque joueur affronte automatiquement l'équipe d'un autre joueur choisi aléatoirement.

Durant cette phase, les joueurs ne peuvent plus modifier leur équipe. En revanche, ils peuvent toujours effectuer des actions n'ayant aucun impact sur le combat en cours, comme gérer leur banc ou acheter des unités dans le magasin. À la fin du combat, le joueur vaincu perd un nombre de point de vie calculé selon la formule suivante : Points de vies perdus = nombre d'unités en vie chez l'adversaire + le niveau de l'adversaire.

À la fin de chaque combat, tous les joueurs gagnent 5 pièces d'or et 2 points d'expérience. Le vainqueur reçoit en plus un bonus de 2 pièces d'or.

La partie prend fin lorsqu'il ne reste plus qu'un seul joueur ayant encore des points de vie.

1 Introduction

1.1 Contexte motivation

Ce projet de bachelor s'inscrit dans la volonté de réaliser un travail à la fois exigeant et en accord avec l'orientation professionnelle envisagée à l'issue des études, à savoir le domaine du jeu vidéo. Il constitue une opportunité de concevoir un jeu de manière structurée et professionnelle, et de disposer ainsi d'une réalisation concrète illustrant les compétences acquises durant le cursus.

1.2 Description du sujet

Ce travail de bachelor consiste en la conception et le développement d'un jeu multijoueur accessible via le web, s'inscrivant dans le genre **auto-battler**. Ce type de jeu oppose plusieurs joueurs (généralement huit) au sein d'une même partie, chacun devant constituer et faire évoluer une équipe d'unités sur un plateau de jeu qui lui est propre. Contrairement à un jeu de stratégie traditionnel, le joueur ne contrôle pas directement ses unités pendant les combats : ceux-ci se déroulent de manière automatique, une fois les phases de préparation (achat, positionnement, équipement) terminées. Une partie se déroule par rounds successifs, alternant une phase de préparation et une phase de combat, durant laquelle chaque joueur affronte tour à tour différents adversaires, jusqu'à ce qu'il ne reste qu'un seul joueur en vie.

Le projet repose sur le développement d'une application complète intégrant une interface utilisateur, une logique métier, et un système de persistance des données. Il s'inscrit dans une démarche de conception logicielle moderne, à travers une architecture distribuée permettant de séparer clairement les différentes responsabilités du système.

Au-delà de l'aspect ludique, ce projet vise à mobiliser un ensemble de compétences acquises durant le cursus, telles que le développement web, la conception d'une architecture multi-tiers, la communication entre client et serveur, ainsi que la mise en place de bonnes pratiques de développement. Une attention particulière est portée à la qualité du code, à la maintenabilité de l'application, ainsi qu'à l'automatisation des tests.

1.3 Architecture du projet

Avant de développer un jeu pensé spécifiquement pour le web, l'accent est mis sur le développement d'une application robuste, pouvant être exploitée dans un contexte web, ce qui implique un certain nombre de choix architecturaux détaillés ci-dessous.

L'application repose sur une architecture multi-tiers, structurée en trois composants principaux : un frontend, un backend, et une base de données.

Le frontend ne contient aucune logique métier. Son unique rôle est d'afficher les différents états de l'application, à savoir l'écran de connexion, le lobby une fois l'utilisateur authentifié, et le déroulement de la partie en cours.

Le backend contient l'intégralité de la logique métier du jeu. Il a pour rôle de simuler chaque combat et d'en transmettre le résultat au frontend afin que celui-ci puisse l'afficher au joueur. Il est également responsable de la validation des entrées utilisateur, ainsi que de la gestion des connexions avec le client.

La base de données contient l'ensemble des informations relatives aux utilisateurs et au déroulement du jeu, notamment l'historique des parties.

Dans une optique de bonnes pratiques de développement, un pipeline d'intégration continue (CI) est également mis en place, afin d'automatiser les phases de test.

1.4 Planification

Pour la planification initiale, l'objectif était de travailler en cascade tout du long. Cependant, ce plan a évolué lors de la réalisation de ce projet. En effet, travailler en cascade a été planifié jusqu'à la réalisation d'une version minimal, suite à quoi une implémentation par itération sera utilisée pour les fonctionnalités supplémentaires.

	Semaine de 15h																	Semaine de 45h					
Étape/semaine	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Étape1 : setup				C																			
cahier des charges				R																			
planning				U																			
Étape 2 : analyze				N																			
analyze des besoins				C																			
spécifications des besoins				H																			
Étape 3: Conception																							
architectural design																							
database design																							
interface design																							
Étape 4: Implémentation des must have																							
Étape 5: Implémentation des should have																							
Étape 6: Finition des documents a rendre																							

Fig. 1. – Plannification

2 State of the art

2.1 Introduction

Dans le cadre de ce projet, dont l'objectif est de développer un jeu selon des standards professionnels, ce travail s'intéresse à la manière dont différents acteurs de l'industrie répondent aux enjeux liés à ce type de développement.

Le projet repose notamment sur l'utilisation du framework Play [1], qui propose déjà un ensemble de solutions intégrées. L'objectif est donc d'analyser les choix technologiques de solutions existantes ainsi que les fonctionnalités offertes par ce framework, afin de concevoir une infrastructure cohérente et réfléchie.

Cette étude portera plus particulièrement sur les aspects suivants : le frontend, l'authentification des utilisateurs, les communications client-serveur, l'intégration entre la base de données et le backend, ainsi que la mise en place d'un pipeline CI/CD.

2.2 Frontend

Ce projet nécessite le développement d'un frontend ne contenant que très peu de logique métier. Son rôle principal est d'afficher un grand nombre de sprites en deux dimensions, de permettre l'interaction avec l'utilisateur, et de transmettre des messages au backend. En effet, l'ensemble des combats est simulé et calculé côté serveur, qui constitue l'unique source de vérité ; le backend transmet ensuite les résultats de la simulation au frontend, chargé de leur affichage.

Le framework Play propose des solutions telles que les vues Scala et les templates Twirl [2] pour le rendu de pages HTML. Toutefois, en raison du niveau d'interactivité requis, cette approche ne semble pas adaptée ; une solution basée sur JavaScript côté client apparaît donc plus pertinente.

Dans l'écosystème JavaScript, plusieurs approches sont envisageables. Il est possible d'utiliser du JavaScript pur, ou des technologies bas niveau telles que WebGL [3] pour gérer le rendu graphique. Des solutions plus haut niveau existent également, comme les bibliothèques de rendu (par exemple PixiJS [4]) ou des moteurs de jeu complets tels que Phaser [5].

L'utilisation de JavaScript pur ou de WebGL offre une grande flexibilité, mais implique un coût de développement important [6]. Compte tenu des contraintes de temps du projet, ces solutions ne sont pas retenues, bien que WebGL puisse constituer une option intéressante dans une perspective d'évolution vers la 3D.

L'un des critères de choix concerne d'ailleurs la compatibilité avec un éventuel rendu 3D. Certaines technologies, notamment les bibliothèques de rendu 2D, ne permettent pas une telle évolution, ce qui pourrait nécessiter une refonte importante en cas de changement futur.

Le choix se restreint ainsi entre une bibliothèque de rendu et un moteur de jeu. La principale différence réside dans les fonctionnalités fournies : un moteur de jeu intègre des éléments complets tels que la boucle de jeu, la gestion du son ou des interactions, tandis qu'une bibliothèque de rendu se concentre principalement sur l'affichage graphique.

Dans le cadre de ce projet, une grande partie des fonctionnalités offertes par un moteur de jeu ne sont pas utiles, notamment la gestion des collisions, la logique métier étant entièrement gérée côté backend.

Selon des benchmarks existants [7], [8], PixiJS apparaît comme l'une des solutions les plus performantes en moyenne, toutes librairies et navigateurs confondus.

Le premier benchmark [8] a été réalisé sur un MacBook Pro, sur plusieurs navigateurs, en affichant huit mille sprites simultanément. Sur Chrome, PixiJS atteint 60 images par seconde et se classe premier ; sur Firefox, il atteint 48 images par seconde et se classe deuxième ; sur les autres navigateurs testés, ses performances se situent dans la moyenne, autour de 24 images par seconde.

Le second benchmark [7] permet de tester soi-même différentes librairies graphiques, en choisissant librement le nombre de sprites à afficher. Lors de nos propres essais, seules PixiJS, Three.js, Phaser et Excalibur sont parvenues à afficher 10'000 sprites à 120 images par seconde. Les statistiques globales fournies par l'auteur du site indiquent par ailleurs des moyennes de

47 images par seconde pour PixiJS et de 43 images par seconde pour Phaser, ce qui confirme l'avantage de PixiJS pour ce cas d'usage.

Le choix de PixiJS permet ainsi d'optimiser les performances d'affichage. Cette approche nécessite toutefois l'ajout de fonctionnalités complémentaires, telles que la gestion du son ou de certaines interactions ; ces besoins peuvent être couverts par des bibliothèques externes, comme Howler.js pour la gestion audio. Ce choix implique également qu'un passage ultérieur à un affichage 3D nécessiterait une refonte importante, PixiJS ne prenant en charge que la 2D.

2.3 Authentication

Afin de proposer une expérience personnalisée, le jeu nécessite la mise en place d'un système d'authentification permettant d'associer des données aux utilisateurs.

Le framework Play propose des mécanismes d'authentification intégrés. Leur utilisation implique toutefois de gérer plusieurs aspects sensibles, tels que le stockage sécurisé des mots de passe (hachage), la gestion des connexions, ainsi que le traitement des erreurs associées.

Une analyse des pratiques adoptées par des applications web populaires, telles qu'Agar.io, GeoGuessr ou encore GeoGamer, montre que l'authentification déléguée constitue une approche largement répandue. Celle-ci repose généralement sur des protocoles standards comme OAuth2 [9], souvent complétés par OpenID Connect, permettant de déléguer l'authentification à des fournisseurs tiers tels que Google ou GitHub.

Cette approche présente plusieurs avantages : elle permet de réduire la complexité de développement, d'améliorer la sécurité en s'appuyant sur des acteurs spécialisés, et d'offrir une meilleure expérience utilisateur grâce à des mécanismes de connexion simplifiés.

Ainsi, le recours à une solution d'authentification déléguée permettrait de concentrer les efforts de développement sur les fonctionnalités principales du jeu, tout en limitant les risques liés à la gestion directe de données sensibles.

2.4 Connexion client-serveur

Le serveur backend doit établir des connexions avec les utilisateurs afin de pouvoir échanger des messages avec ces derniers. C'est grâce à ces connexions que la communication entre le client et le serveur est assurée. Il doit également assurer le maintien de ces connexions, et être capable de les rétablir en cas d'interruption.

Compte tenu du volume et de la fréquence des échanges, un canal de communication bidirectionnel persistant, tel que les WebSockets, constitue une solution plus adaptée que des approches traditionnelles comme le polling HTTP [10], utilisé avant l'apparition des WebSockets et impliquant des requêtes répétées du client vers le serveur, ce qui engendre une surcharge inutile.

Une autre alternative est l'utilisation des Server-Sent Events (SSE) [11], qui permettent au serveur d'envoyer des mises à jour au client. Cette solution reste toutefois unidirectionnelle et ne permet pas au client de communiquer directement avec le serveur, ce qui la rend inadaptée dans le cadre d'un jeu interactif.

Le framework Play propose une intégration native des WebSockets [12] pour gérer les communications entre le backend et les utilisateurs, avec deux approches principales disponibles : l'une basée sur le modèle d'acteurs [13], l'autre utilisant directement Akka Streams pour manipuler les flux de données.

Le choix s'oriente vers l'approche basée sur le modèle d'acteurs, particulièrement adapté à la gestion d'états concurrents. Il permet notamment d'associer un acteur à chaque client afin de gérer son état individuel, ainsi qu'un ou plusieurs acteurs supplémentaires pour représenter l'état global de la partie.

2.5 Gestion des connexions

Pour assurer le maintien des connexions, il sera nécessaire de mettre en place un système de heartbeat, ainsi qu'un mécanisme de reconnexion en cas de déconnexion.

Concernant le système de reconnexion, l'objectif principal est de pouvoir transmettre l'état actuel de la partie au moment où l'utilisateur se reconnecte. Cela nécessite de conserver un historique de l'état de la partie, cohérent entre tous les utilisateurs. Il s'agit d'un problème classique de concurrence en environnement distribué.

Certains aspect restent sujet à débat, notamment la taille de l'historique à conserver, ainsi que la manière dont la reconnexion sera présentée à l'utilisateur : les combats manqués doivent-ils être rejoués séquentiellement, ou seul le résultat final doit-il être affiché, ou encore est-ce que l'utilisateur doit recevoir le détails des événements survenus durant sa déconnexion ?

2.6 Lien entre la base de données et le backend

L'application devra stocker des données relatives aux utilisateurs, qu'il faudra créer, modifier ou les utiliser. Des requêtes adaptées seront donc nécessaires pour accéder à ces données et faire le lien entre le backend Play et la base de données PostgreSQL. Plusieurs solutions sont envisageables pour cela : l'écriture de SQL pur, l'utilisation de JOOQ [14], ou encore le recours à un ORM(Object-Relational Mapping).

Le premier problème rencontré est celui de l'impedance mismatch, c'est-à-dire la dissonance entre le modèle orienté objet utilisé dans le backend et le modèle relationnel de la base de données : le premier repose sur des notions telles que l'héritage ou les références directes entre objets, quand le second ne connaît que des tables plates reliées par des clés étrangères. Pour résoudre ce problème, la solution la plus simple et la plus économique consiste à utiliser un ORM, dont le rôle principal est justement de gérer cette différence. Play propose d'ailleurs des tutoriels d'intégration pour des ORM tels qu'Hibernate [15], largement utilisé dans l'industrie.

L'abstraction proposée par les ORM s'avère cependant souvent imparfaite (on parle d'abstraction « leaky », c'est-à-dire laissant transparaître les détails du fonctionnement sous-jacent) : ces outils ne constituent donc pas une solution à l'ensemble des problèmes rencontrés. Leur utilisation nécessite une attention particulière afin de préciser au mieux les intentions du développeur ; sans cela, l'ORM peut générer des structures inefficaces, telles que des tables inutiles, ou entraîner des problèmes de performance dans les requêtes générées.

Les ORM restent néanmoins une bonne solution pour la plupart des requêtes liées à un utilisateur particulier. En revanche, dans le cas d'un système de classement (leaderboard) ou de statistiques portant sur l'ensemble des personnages, des requêtes plus complexes seront nécessaires. Dans ce type de situation, l'écriture de SQL pur ou l'utilisation de JOOQ pourrait s'avérer plus appropriée.

2.7 CI/CD

Ce projet ayant pour objectif la mise en place d'un pipeline CI/CD, cette section s'intéresse aux différentes solutions disponibles.

Afin de garantir un environnement standardisé pour les différents builds tout au long du projet, Docker et son système d'images ont été utilisés.

L'étude des outils employés par différents acteurs du domaine montre que Jenkins constitue une solution populaire, en raison de sa flexibilité et de son caractère open source. Il permet de ne pas tout reconstruire soi-même, grâce à l'utilisation de plugins créés et maintenus par une large communauté. Son utilisation demande cependant un investissement en temps conséquent, et nécessiterait la mise en place d'un serveur dédié pour l'exécuter. Les retours de développeurs l'ayant utilisé indiquent en outre que la maintenance des pipelines peut devenir chronophage, notamment en raison de problèmes de compatibilité entre certains plugins.

Une alternative plus simple à mettre en œuvre est GitHub Actions. Cette solution est particulièrement pratique, puisque le versionnement du projet a pour objectif d'être réalisé sur GitHub ; elle permet une intégration rapide et ne nécessite pas de serveur supplémentaire pour exécuter les pipelines, sauf en cas d'utilisation de runners auto-hébergés.

Bien qu'il existe des tutoriels réalisés par Riot Games (développeurs de Teamfight Tactics) présentant une intégration basée sur Jenkins et Docker [16], nous avons choisi d'utiliser GitHub Actions afin de gagner du temps et de simplifier la mise en place du pipeline. Seule une partie de Livraison continue sera mise en place dans un premier temps.

3 Architecture logicielle

Après avoir analysé les différentes solutions à notre disposition, nous allons maintenant présenter l'architecture logicielle retenue, ainsi qu'une brève présentation des technologies associées.

3.1 Schéma de l'architecture

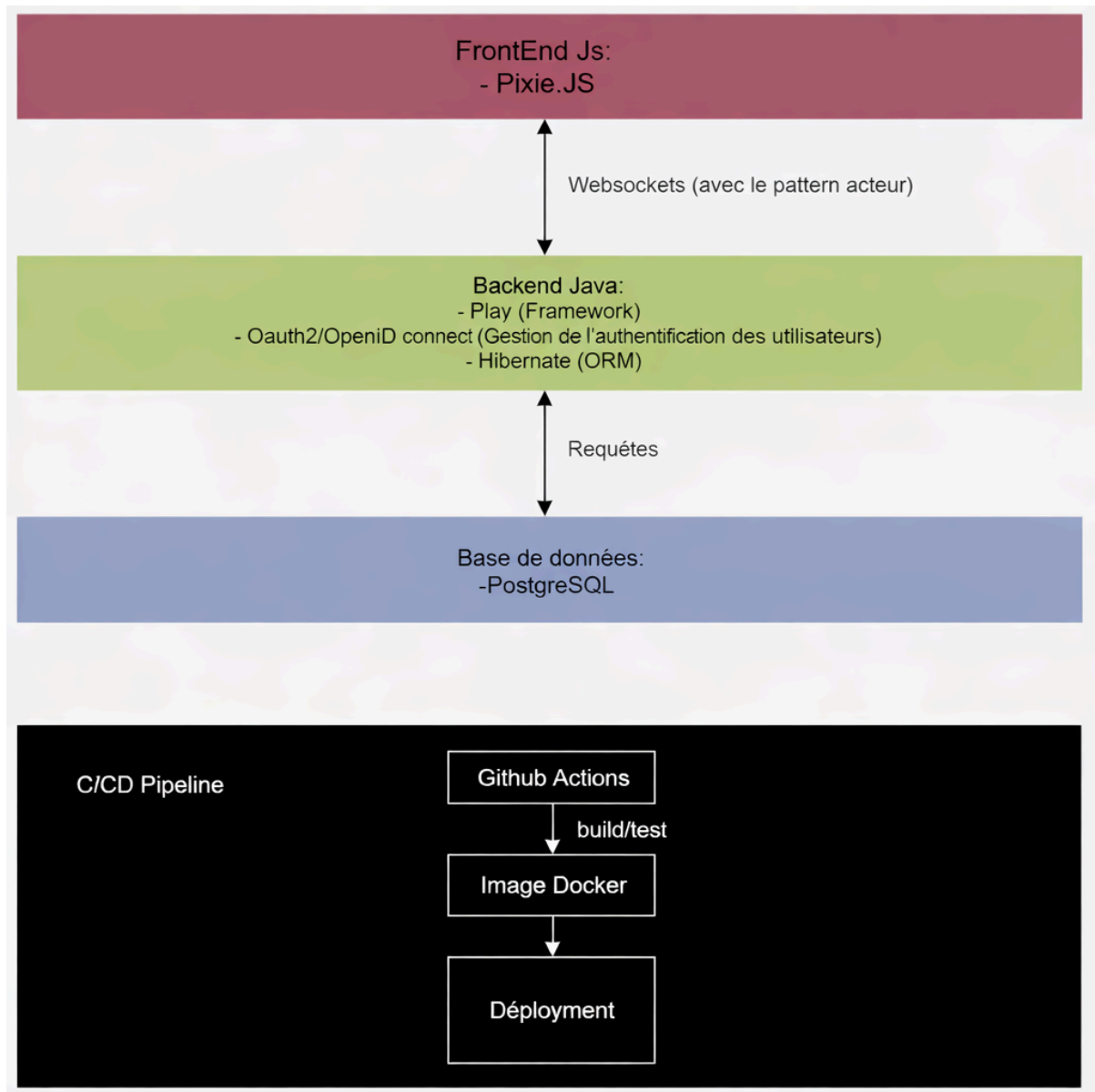


Fig. 2. – Schéma architecture logiciel

3.2 Frontend

Le frontend, c'est-à-dire l'interface client, est développé en JavaScript, notamment à l'aide de la librairie PIXIJS.

3.2.1 PIXIJS

PIXIJS est un moteur de rendu 2D open source permettant de créer des visuels de haute qualité sur le web à l'aide de JavaScript. Ce moteur est construit sur WebGL et, de manière optionnelle, sur WebGPU, ce qui lui permet d'offrir des performances élevées et d'exécuter sans difficulté des scènes contenant de nombreux sprites.

Dans le cadre de ce projet, PIXIJS gère l'affichage de nombreux sprites, tels que les personnages, l'interface utilisateur, ainsi que l'ensemble des animations de combat. Il permet ainsi de supporter un grand nombre d'éléments graphiques simultanés sans dégradation significative des performances.

3.3 Backend

Le backend est développé en Java. Il exécute l'ensemble de la logique métier de l'application, notamment l'authentification, la gestion des connexions avec les utilisateurs, la gestion des parties, ainsi que la simulation des combats.

3.3.1 Play

Le framework Play est un framework web open source conçu pour le développement d'applications web en Java et Scala. Il repose sur une architecture moderne basée sur le modèle MVC (Model-View-Controller), ce qui permet de structurer clairement le code et d'en faciliter la maintenance. Play se distingue par son approche réactive et non bloquante, qui lui permet de gérer efficacement un grand nombre de requêtes simultanées. Il est également fortement intégré à l'écosystème Java, ce qui le rend compatible avec de nombreuses bibliothèques et outils utilisés en entreprise. Dans le cadre de ce projet, le framework Play est utilisé pour

implémenter la partie backend de l'application, notamment la gestion des connexions à l'aide du pattern acteur et de Pekko Streams, permettant de gérer la concurrence de manière fiable.

3.3.2 OAuth2 / OpenID Connect

OAuth 2.0 est un protocole d'autorisation largement utilisé, qui permet à une application d'accéder à des ressources protégées au nom d'un utilisateur, sans avoir besoin de connaître ses identifiants. Il repose sur l'émission de tokens d'accès, définissant des permissions limitées et temporaires. OpenID Connect est une couche d'authentification construite au-dessus d'OAuth 2.0. Elle permet de vérifier l'identité d'un utilisateur et de récupérer des informations de profil de manière standardisée, à l'aide d'un mécanisme basé sur des tokens d'identité (ID Token). Dans le cadre de ce projet, si OAuth2 n'est pas implémenter pour quelconque soucis il est prévu de gérer l'authentification nous même.

3.3.3 Hibernate

Hibernate est un framework de type ORM (Object-Relational Mapping) utilisé en Java pour faciliter la persistance des objets en base de données relationnelle. Il permet de faire le lien entre le modèle objet de l'application et le modèle relationnel de la base de données, en automatisant la génération des requêtes SQL. Grâce à Hibernate, il est possible de manipuler directement des objets Java sans avoir à écrire explicitement la majorité des requêtes SQL. Le framework se charge de la traduction entre les objets et les tables, ainsi que de la gestion des relations, des transactions et du cycle de vie des entités. Dans le cadre de ce projet, Hibernate est utilisé pour simplifier l'accès à la base de données et réduire la complexité liée à l'écriture et à la maintenance des requêtes SQL.

3.4 Base de données

PostgreSQL est un système de gestion de base de données relationnelle (SGBDR) open source, reconnu pour sa robustesse, sa conformité aux standards SQL et ses performances. Il permet de stocker, organiser et interroger des données structurées à l'aide du langage SQL. PostgreSQL offre de nombreuses fonctionnalités avancées, telles que la gestion des transactions ACID, le

support d'index performants, ainsi que la possibilité de définir des types de données personnalisés et d'exécuter des requêtes complexes. Il est également apprécié pour sa fiabilité et sa capacité à gérer de grandes quantités de données. Dans le cadre de ce projet, PostgreSQL est utilisé comme base de données principale afin de stocker les informations liées aux utilisateurs et aux données applicatives, tout en garantissant la cohérence et l'intégrité des données.

3.5 Pipeline CI/CD

L'objectif du pipeline CI/CD mis en place est d'automatiser les étapes de construction (build) et de test de chaque nouvelle sortie (release), tout en garantissant l'absence de mauvaises surprises grâce à l'exécution des tests dans un environnement identique à celui utilisé en production. Seule une démarche d'Intégration continue est mise en place à ce stade, le déploiement, lui, reste manuel.

3.6 GitHub Actions

GitHub Actions est une plateforme d'intégration et de déploiement continu (CI/CD) intégrée directement à GitHub. Elle permet d'automatiser des workflows tels que les tests, la compilation, le linting, ou encore le déploiement d'une application à chaque modification du code source. Les workflows GitHub Actions sont définis sous forme de fichiers YAML et s'exécutent automatiquement en réponse à des événements du dépôt, tels qu'un push, une pull request, ou la création d'une release. Cette approche permet de standardiser et de fiabiliser les processus de développement. Dans le cadre de ce projet, GitHub Actions est utilisé pour automatiser les tests et les étapes de déploiement, afin de garantir la qualité du code tout en simplifiant la mise en place du pipeline CI/CD.

3.7 Docker

Docker est une plateforme open source permettant de créer, déployer et exécuter des applications dans des conteneurs. Un conteneur est un environnement isolé regroupant une application ainsi que l'ensemble de ses dépendances, garantissant un comportement identique quel que soit

le système hôte. Grâce à cette approche, Docker simplifie la gestion des environnements de développement, de test et de production, en réduisant les problèmes liés aux différences de configuration entre machines. Les conteneurs sont légers, rapides à démarrer, et plus efficaces en ressources que les machines virtuelles traditionnelles. Dans le cadre de ce projet, Docker est utilisé afin de standardiser les environnements d'exécution et de faciliter le déploiement des différentes parties de l'application, notamment le backend et les services associés.

4 Conception

Cette section présente les choix de conception réalisés lors du développement du projet. Avant d'aborder les aspects techniques liés à l'implémentation. C'est une étape primordiale afin d'avoir de bonnes bases pour le projet.

Dans un premier temps, la conception orientée objet est présentée afin de décrire les principales classes du projet, leurs responsabilités et les relations qu'elles entretiennent entre elles.

La section s'intéresse ensuite au modèle de données, qui décrit la manière dont les informations sont représentées et persistées. Les choix effectués en matière de stockage des données, notamment l'approche retenue pour conserver l'état des parties, y sont également justifiés.

Enfin, les principaux mécanismes de l'application sont détaillés, notamment l'architecture client-serveur et le fonctionnement du système de combat.

4.1 Conception Orientée Objet

4.1.1 Controller

Avant toute chose, afin d'améliorer la lisibilité des schémas, les accesseurs (**getters** et **setters**) ainsi que les constructeurs ne seront pas représentés. En effet, la majorité des constructeurs utilisent l'injection de dépendances et leur présence alourdirait considérablement les diagrammes sans apporter d'informations pertinentes.

La base de notre application Play est le **HomeController**, qui est chargé de répondre aux requêtes des utilisateurs et de les rediriger via les différentes routes de l'application.

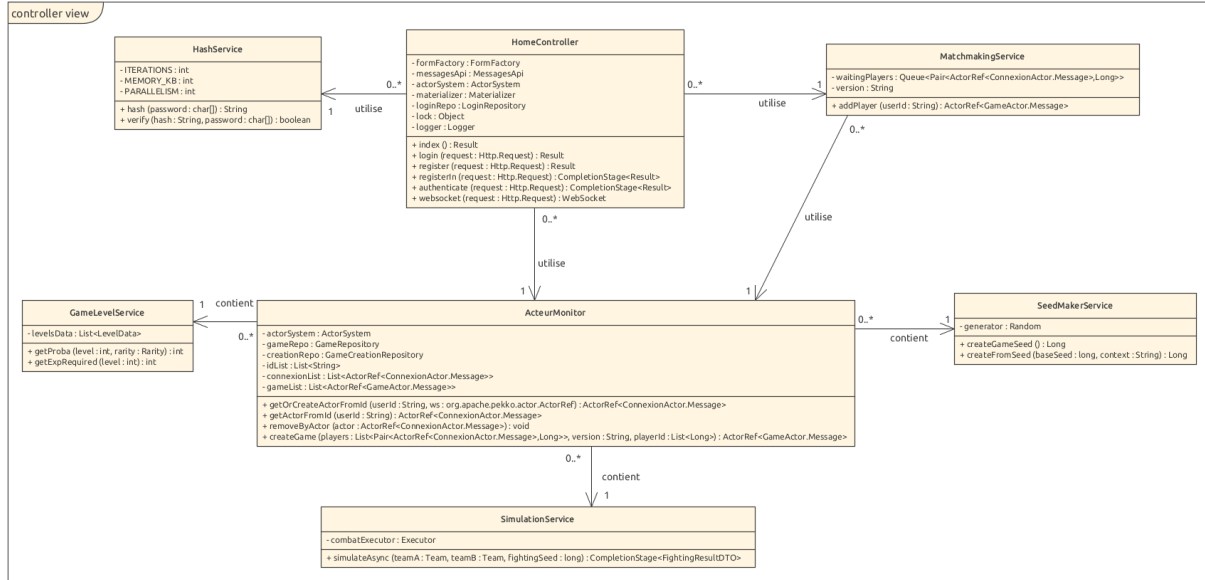


Fig. 3. – UML du HomeController

Ce contrôleur a pour objectif de gérer toute la partie liée à l'authentification, en s'appuyant sur des formulaires ainsi que sur un service de hachage des mots de passe, qui sera détaillé ultérieurement. Son rôle est également de créer une WebSocket, puis de déléguer la gestion des connexions à l'**ActorMonitor** et la gestion du matchmaking ainsi que le lancement des parties au **MatchmakingService**, lorsque toutes les conditions sont réunies.

L'**ActorMonitor** est responsable de la création et de la gestion des acteurs de l'application. Son fonctionnement sera présenté plus en détail dans une section dédiée.

Cette séparation permet d'isoler la gestion du cycle de vie des connexions WebSocket de la logique de matchmaking, qui pourrait évoluer indépendamment: par exemple si l'on souhaite plus tard changer l'algorithme d'appariement des joueurs sans toucher à la gestion des connexions.

Le **GameLevelService** est chargé de stocker les données relatives aux niveaux des joueurs. Comme expliqué précédemment, le niveau d'un joueur détermine le nombre maximal d'unités que son équipe peut posséder. Il influence également les probabilités d'obtenir des unités plus ou moins rares dans la boutique. Ce service centralise donc ces informations ainsi que l'expérience nécessaire pour atteindre le niveau suivant. Ces règles étant identiques pour toutes les parties en cours, les stocker directement sur chaque Team/Game aurait entraîné une duplication inutile.

Le **SeedMakerService** est chargé de générer les **seeds** utilisées pendant une partie afin de maîtriser les mécanismes aléatoires et de garantir un comportement reproductible, mais cela sera expliqué plus en détails lors de l'implémentation.

Enfin, le **SimulationService** est responsable de simuler les combats entre deux équipes afin d'en déterminer le vainqueur.

4.1.2 Unit

Comme expliqué précédemment, les unités sont au coeur du jeu. Les unités (**Unit**) stockent leur statistiques, leur nom, ainsi que leur compétence contenu sous formes de fragment (**AbilityFragment**). Chaque fragment permet a l'aide de sa stratégie de détermine la target de l'effect et de la lui appliquer.

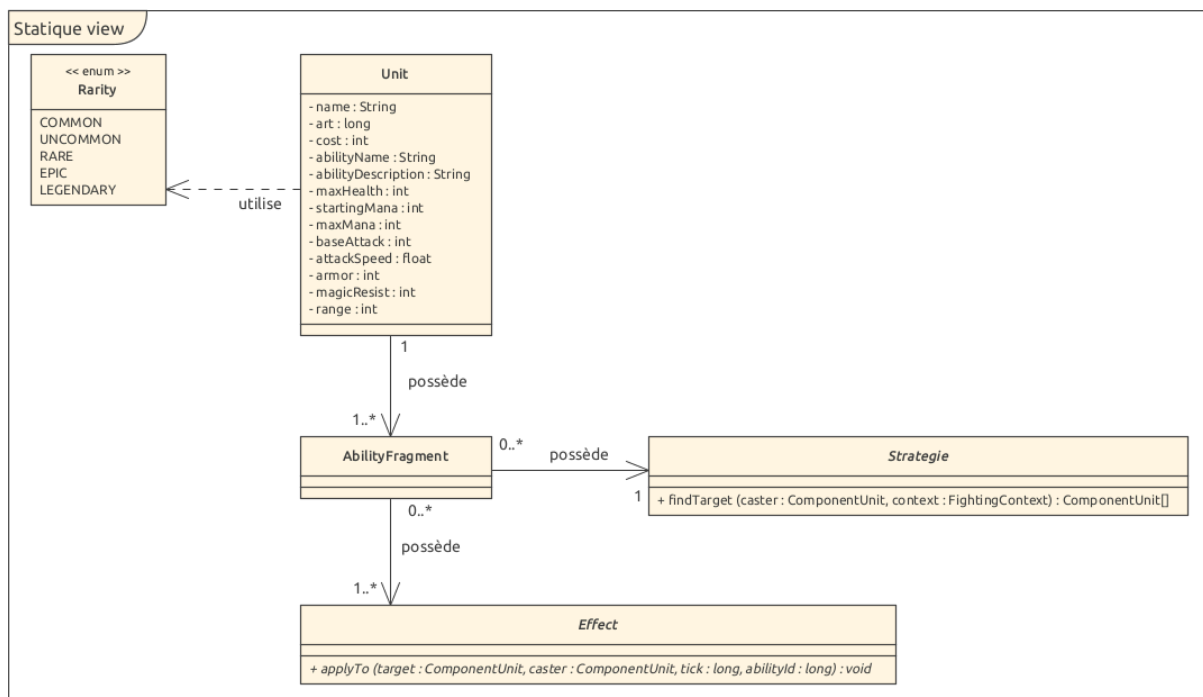


Fig. 4. – UML de l'Unité

La compétence est fragmentée afin que l'on puisse avoir plusieurs type d'effect qui affectent des cibles différentes. Sans cette fragmentation on aurait soit un système de ciblage commun

à tous les effets soit chaque effet doit gérer lui même son ciblage. Avec cette fragmentation et l'aide du pattern Stratégie, il devient possible de décrire un système de ciblage différent pour chaque fragment, offrant une grande modularité.

4.1.3 Effet

Les deux sous-classes d'effet actuelles sont les **scalingEffect** et les **statChangingEffect**. Les **ScalingEffect** sont des effets possédant une puissance de base mais qui gagne en intensité en fonction d'une certaine statistique du lanceur de l'effet. La ou les **StatChangingEffect** n'ont pour but que de changer les statistiques de la cible. C'est notamment à l'aide de ces effets que les objets améliorent les unités.

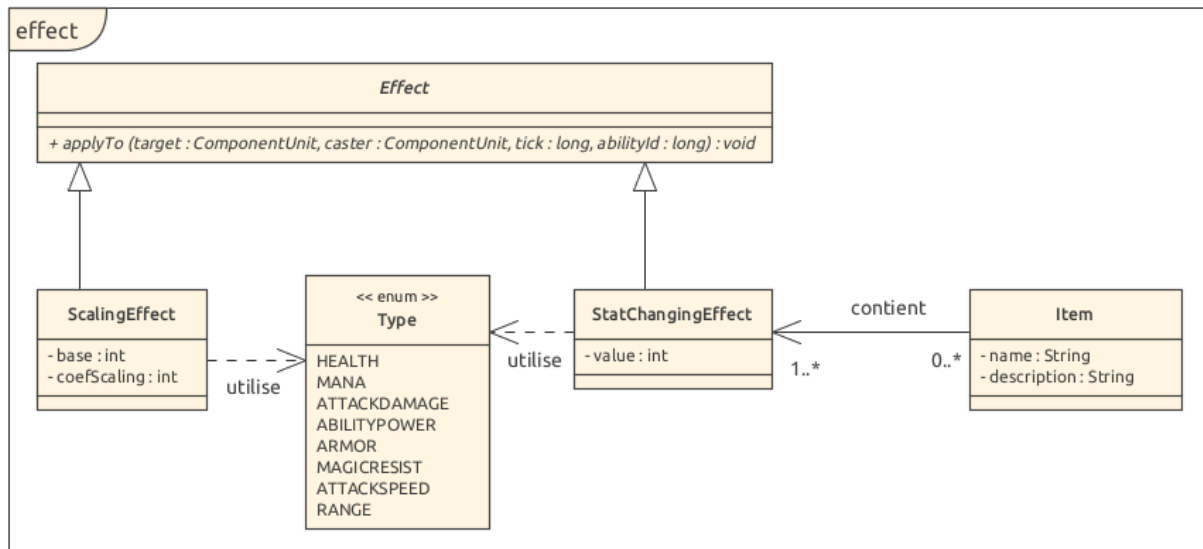


Fig. 5. – UML des Effets

Cette séparation permet d'utiliser le polymorphisme à son plein potentiel afin que chaque sous-classe d'effet traduise un comportement d'effet bien différent des autres.

4.1.4 User

Pour que notre jeu fonctionne il nous faut des utilisateurs qui jouent des parties. On a donc des comptes représentant les utilisateurs (**User**), qui ont une équipe par partie auquel ils ont participé ou participent actuellement. Les équipes (**Team**) ont pour but de changer en fonction des actions de l'utilisateur, ainsi que selon le déroulement de la partie. La partie (**Game**), aura pour but de stocker les équipes et de savoir lesquelles sont disqualifiées ou non.

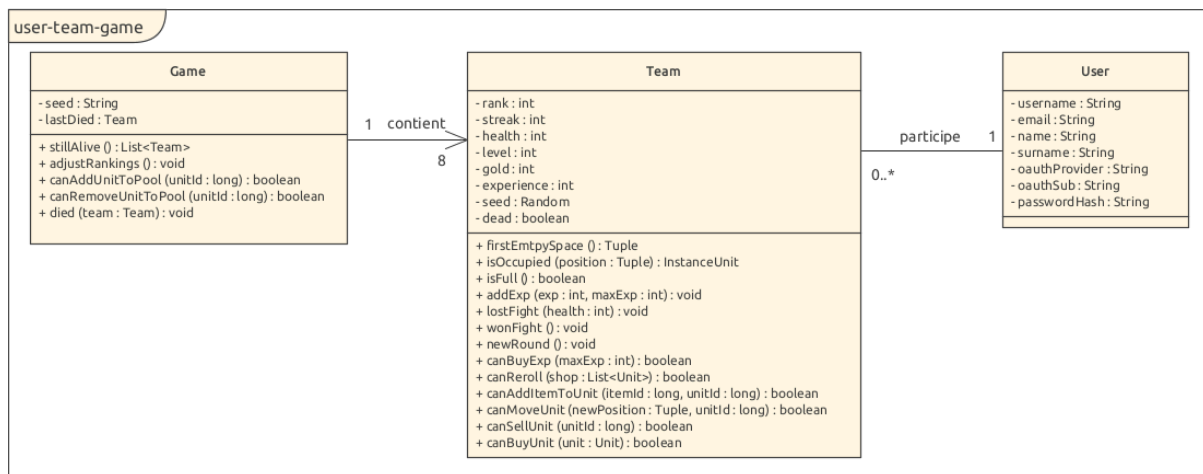


Fig. 6. – UML de l'utilisateur

Un joueur pouvant participer à plusieurs parties, la Team représente sa participation à une partie donnée, distincte de son identité de joueur (User).

4.1.5 Game

La partie a aussi comme objectif de gérer les stocks de champions et de donner une unité aléatoirement provenant d'un stockage. Les champions sont répartis dans différents stockage (**Pool**), chacun différencié par la rareté des champions qu'il stocke. Et ces stockages sont eux aussi séparés en entrées (**PoolEntry**), afin que l'on sache combien de fois de la même unité il reste encore dans ce stockage.

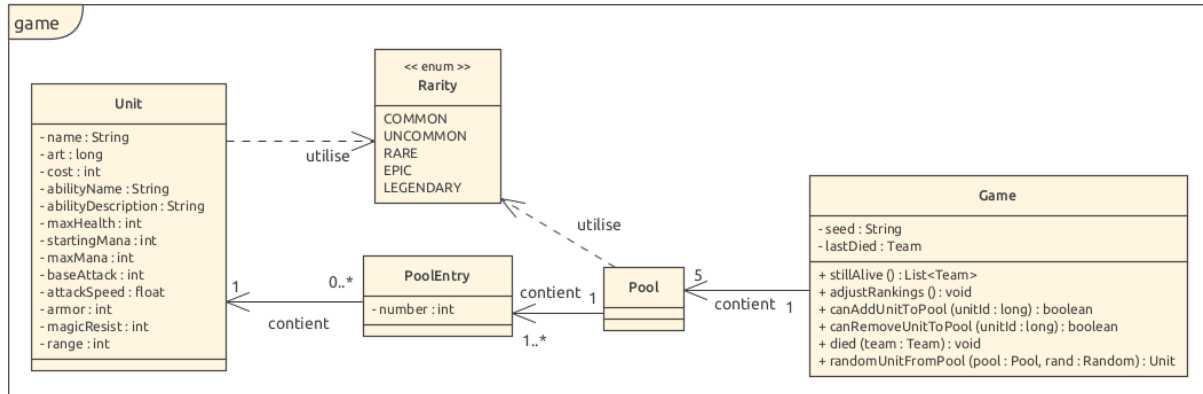


Fig. 7. – UML de la partie

Les **PoolEntry** permettent de simplifier la gestion des unités. Sans eux, il faudrait manipuler directement une liste d'unités et la mettre à jour en permanence.

Comme l'objectif est de conserver au moins un exemplaire de chaque unité en stock, il serait nécessaire de parcourir toute la liste à chaque fois que l'on souhaite connaître le nombre d'unités disponibles. Cette approche est peu pratique et potentiellement coûteuse.

Avec les **PoolEntry**, ce problème disparaît : chaque entrée centralise les informations nécessaires, notamment la quantité disponible, ce qui permet d'obtenir rapidement un décompte sans avoir à parcourir l'ensemble des unités.

4.1.6 Team

Enfin l'équipe doit stocker les champions qui la composent. Pour cela elle utilise des **InstanceUnit** qui stockent l'unité de base ainsi que toutes les données modifiables de l'unité telles que ses objets, sa position ou encore son niveau. L'équipe doit aussi stocker le magasin actuellement proposé au joueur.

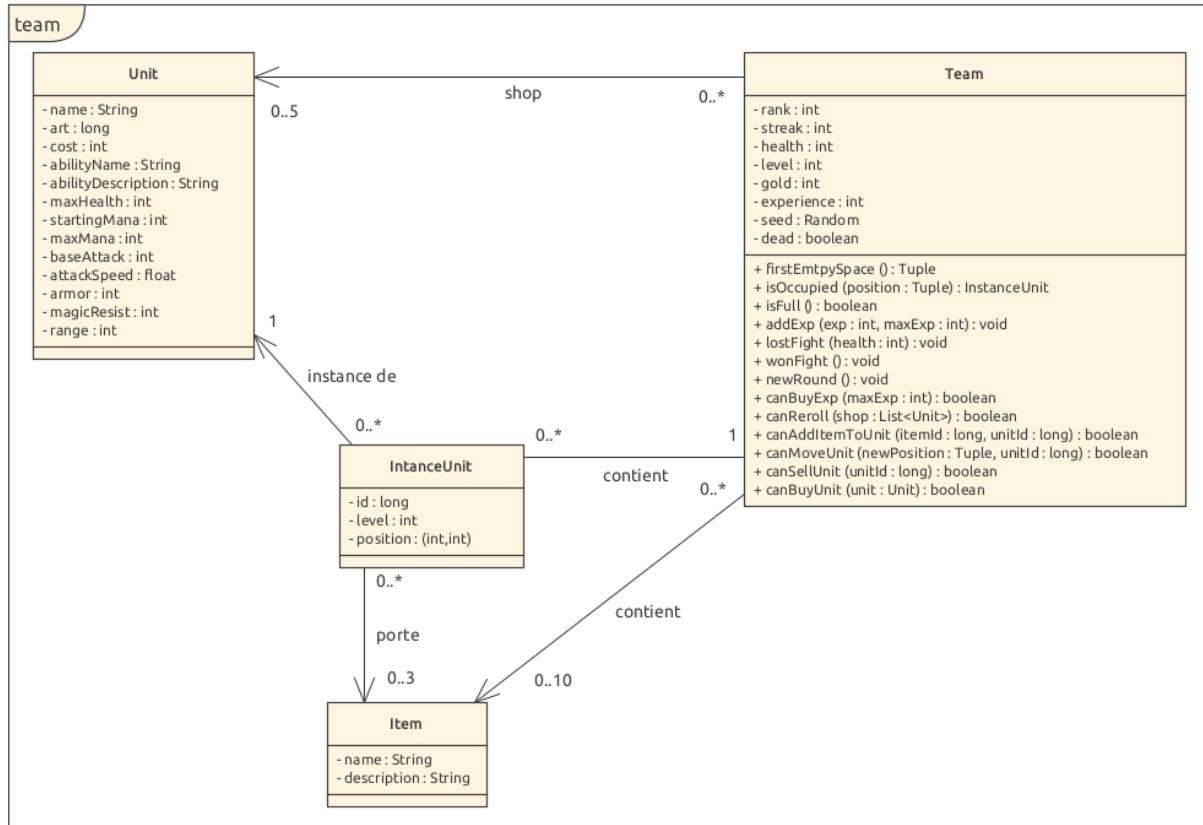


Fig. 8. – UML de l'équipe

L'équipe stocke aussi un snapshot du dernier état du magasin connu en stockant les unités présentes dedans.

L'utilisation d'**InstanceUnit** nous permet d'avoir des unités qui peuvent subir des modifications sans jamais devoir changer l'unité de base, ceci permet aussi d'éviter la duplication des statistiques lorsque l'on possède plusieurs fois la même unité.

4.2 Modèle des données

Afin de pouvoir gérer et faire évoluer nos données sans complexifier excessivement le code applicatif, il a semblé évident de devoir stocker l'ensemble des données de notre application dans une base de données structurée. On y a donc persisté nos unités, ainsi que leurs compétences

et les objets. On y persiste également les parties, les équipes qui y participent, ainsi que les utilisateurs.

4.2.1 Choix de persistance des événements : relationnel normalisé plutôt qu'événementiel en JSON

L'un de nos souhaits était de permettre de rejouer les parties une fois terminé, afin de pouvoir les étudier et s'améliorer, comme c'est l'usage dans la plupart des jeux compétitifs. Une solution basée sur des événements semble naturelle, chaque partie pouvant se traduire simplement comme une suite d'actions amenant un utilisateur d'un état A à un état B. Pour cela, deux approches étaient envisageables :

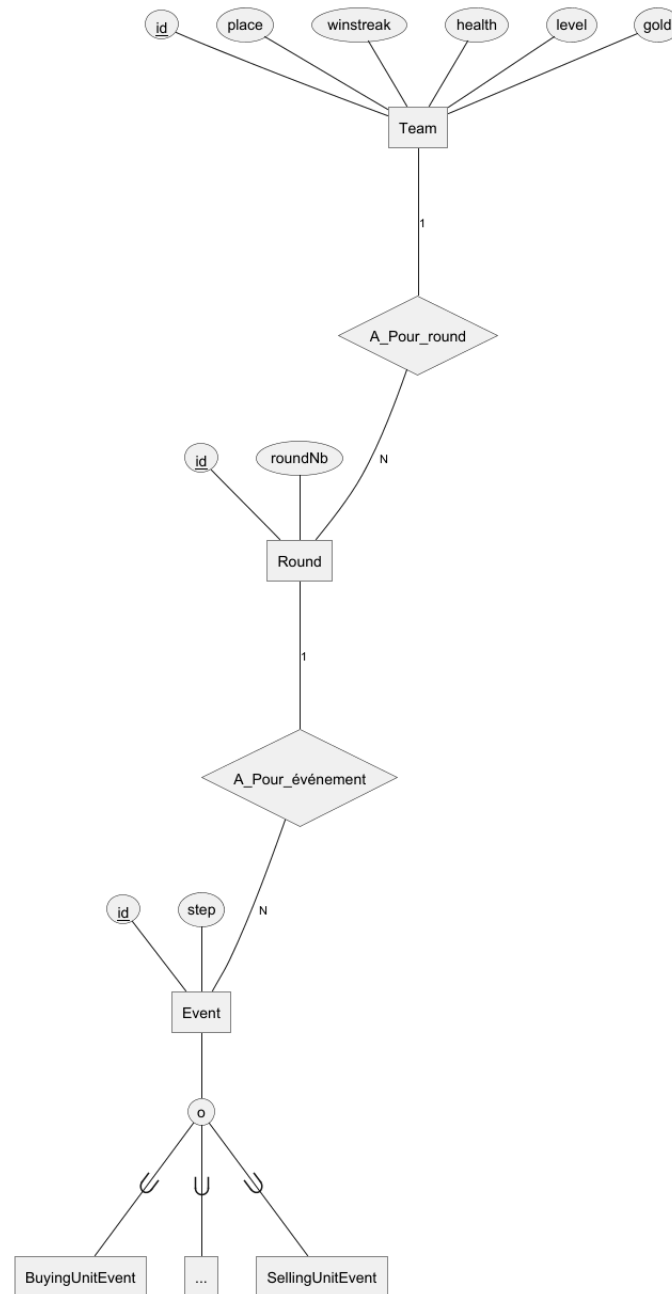


Fig. 9. – Schéma replay sans Json

Un schéma relationnel normalisé, avec une table dédiée pour chaque type d'événement, où chaque colonne est typée et validée par la base de données elle-même.

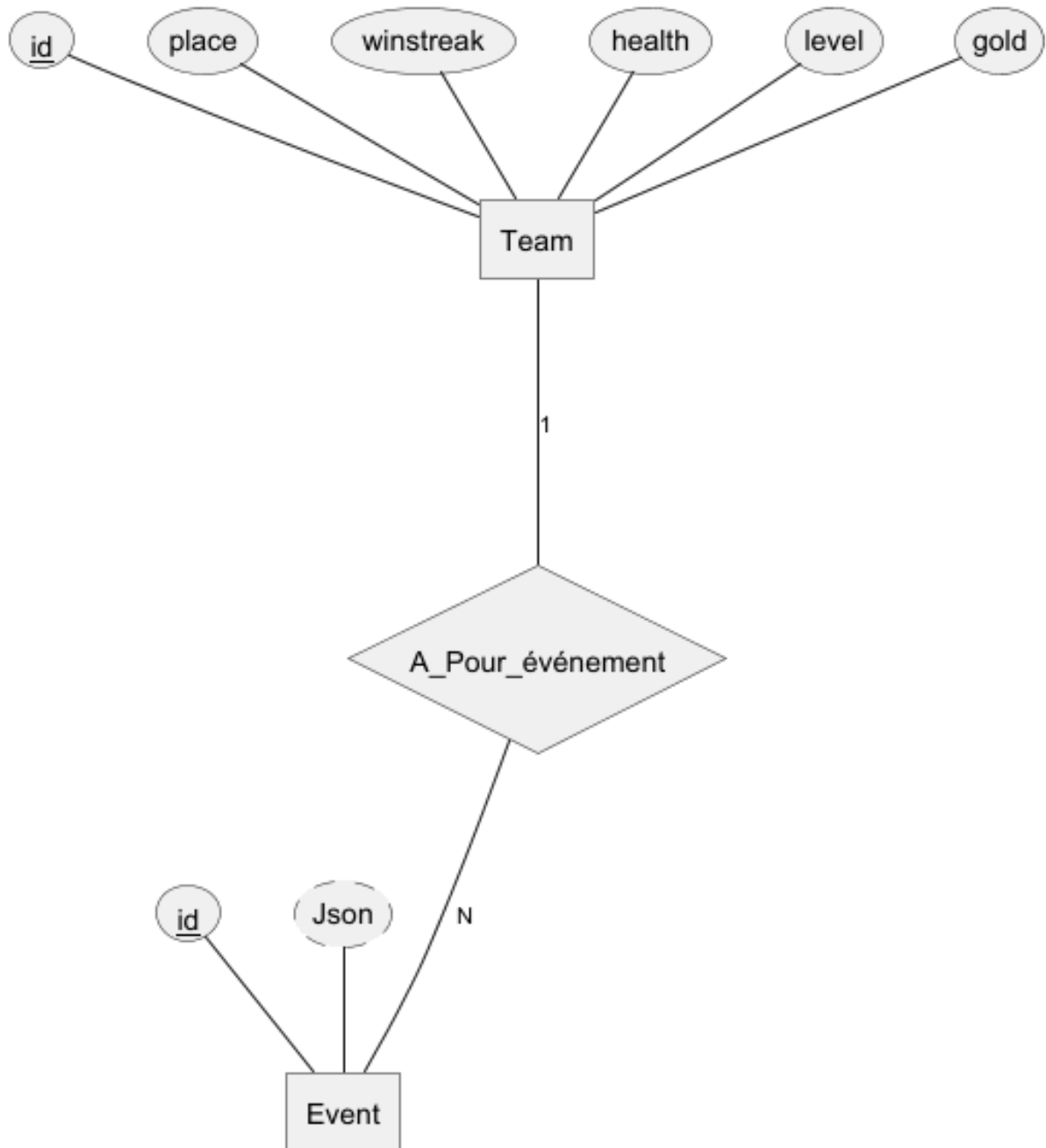


Fig. 10. – Schéma replay avec Json

Un stockage événementiel basé sur du JSON, où chaque événement serait stocké dans une unique table, sous la forme d'un simple champ JSON contenant les détails propres à son type.

Cette deuxième option présentait un avantage réel : une base de données plus compacte, ainsi qu'une conception s'arrêtant à une seule table, sans avoir à modéliser individuellement chaque type d'événement.

Le problème de cette approche est qu'elle nous aurait fait perdre l'intégrité référentielle et la validation structurelle de nos événements, normalement garanties par le moteur de la base de données lorsqu'un schéma est explicitement défini pour chaque table. Il aurait alors fallu réaliser nous-mêmes, côté backend, la validation et le parsing de chaque événement afin de nous assurer de leur cohérence, ce qui aurait été un travail supplémentaire non négligeable, pour un résultat offrant moins de garanties que ce qu'un schéma relationnel strict assure nativement.

On a donc préféré normaliser complètement notre modèle de données plutôt que d'utiliser un champ JSON générique, afin de conserver une validation stricte du schéma de chaque type d'événement directement au niveau de la base de données, plutôt que de devoir réimplémenter cette validation côté backend.

Concrètement, cela signifie que chaque type d'action possible du joueur correspond à sa propre table, héritant d'une table event commune (portant les champs partagés comme l'horodatage et le round concerné), sur le même principe d'héritage relationnel déjà présenté pour nos entités de jeu (Effect, AbilityFragment, etc.). Ne stocker que ce qui n'est pas dérivable autrement permet de garder ces tables légères : par exemple, lors de l'achat d'une unité, seul l'identifiant de l'unité achetée est conservé, son coût restant récupérable depuis la table unit au moment de rejouer l'événement.

4.2.2 Rappel des actions possibles du joueur

Pour mieux comprendre la nature des événements que nous devons persister, il est utile de rappeler le déroulement d'une partie et les actions qu'un joueur peut y effectuer. Une partie se compose d'une succession de rounds, chacun alternant une phase d'achat et une phase de combat. Durant la phase d'achat, le joueur peut notamment acheter, vendre ou déplacer une unité, lui attribuer un objet, relancer son shop, ou encore acheter de l'expérience. Le diagramme de séquence suivant illustre ces différentes actions et leur enchaînement au cours d'un round :

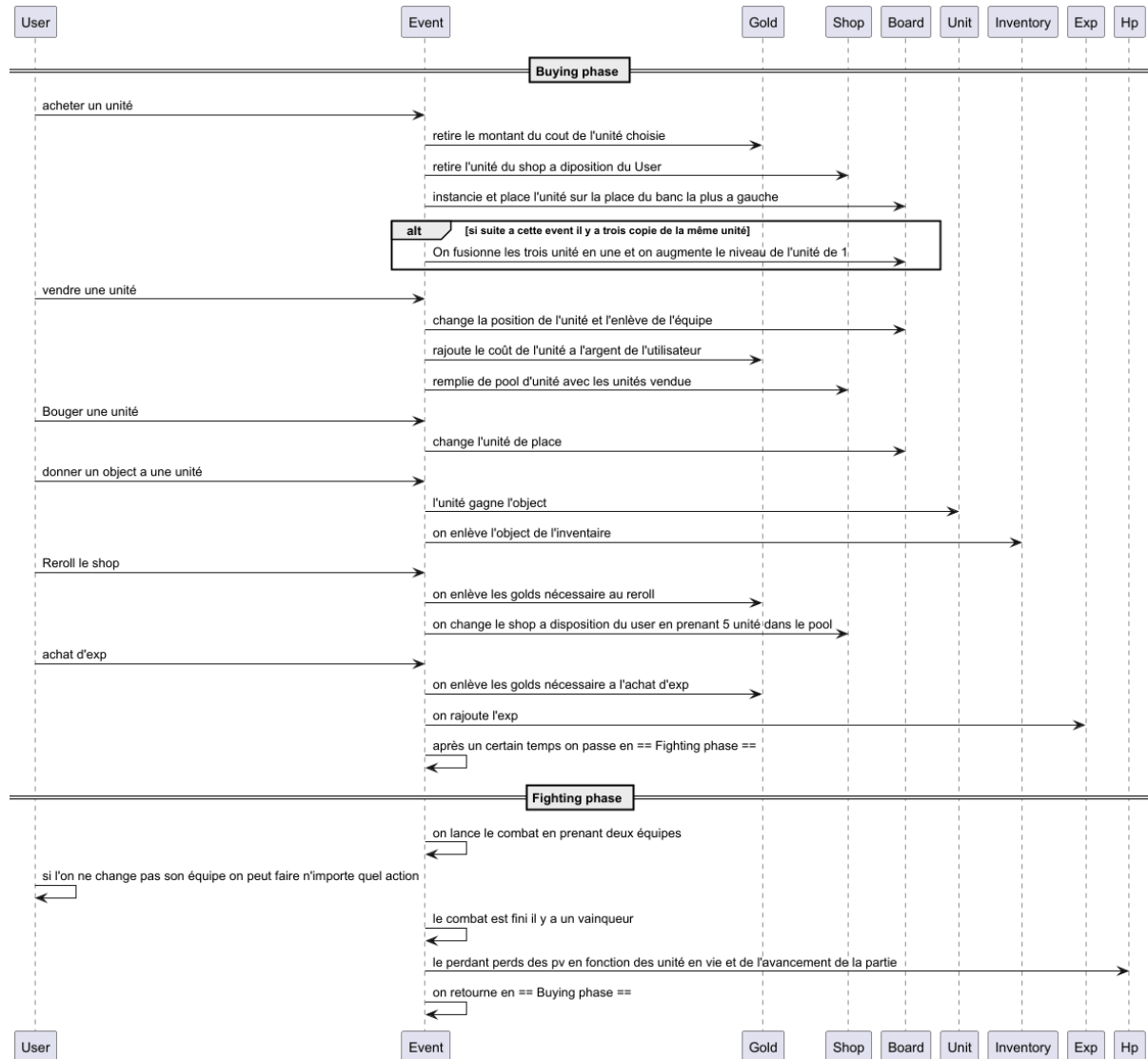


Fig. 11. – Schéma de séquence user input

C'est cette même liste d'actions qui détermine directement les types d'événements que nous persistons en base de données, chacun correspondant à l'une de ces actions.

4.2.3 Modélisation des événements

On distingue trois catégories d'événements, toutes héritant d'une classe commune Event.

4.2.3.1 Événements liés aux données de base du joueur

Une première catégorie regroupe les événements traduisant un changement dans les données fondamentales d'un joueur : ses points de vie, son expérience, ou son or. Chaque Event est associé à un Round, ce qui permet de savoir à quel round un événement donné s'est produit, ainsi qu'à un step, propre à l'équipe concernée, qui permet de connaître l'ordre relatif des événements au sein de ce round. Les combats sont quant à eux représentés par une association entre deux Rounds, traduisant le fait que deux équipes se sont affrontées lors d'un round donné. L'ensemble de ces rounds est finalement rattaché à l'équipe (Team) à laquelle ils appartiennent.

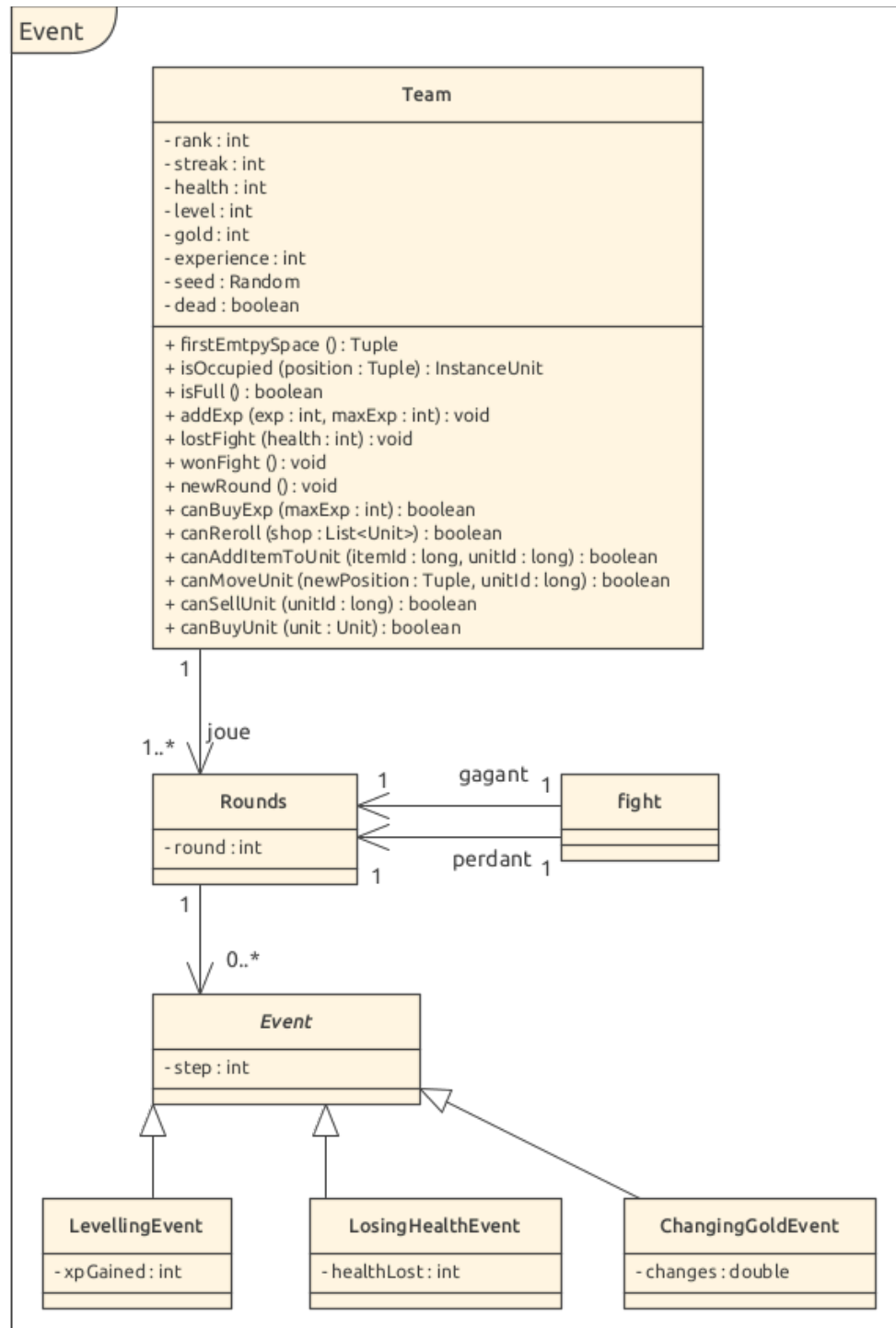


Fig. 12. – UML team/Event

4.2.3.2 Événements liés aux unités

Une deuxième catégorie regroupe l'ensemble des actions du joueur portant sur une unité : achat, vente, déplacement, ou modification des objets qui lui sont attribués. Ces événements sont tous représentés comme des sous-classes d'UnitEvent, elle-même sous-classe d'Event. Chaque UnitEvent référence l'InstanceUnit concernée par l'action, ce qui permet de savoir précisément quelle unité a été affectée par l'événement, tout en bénéficiant des attributs communs (round, step) hérités d'Event.

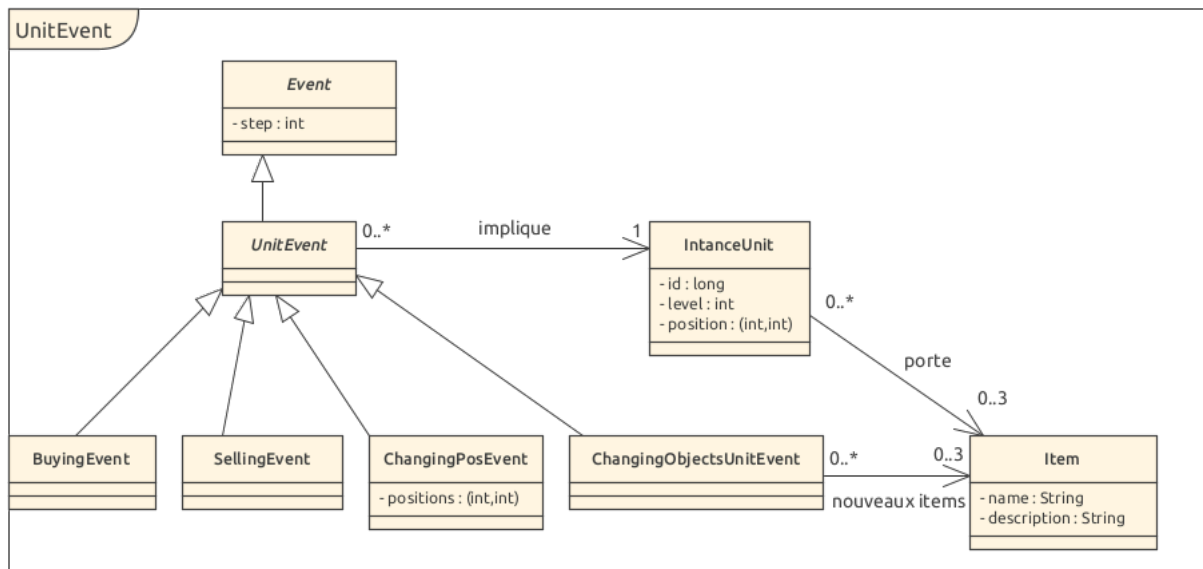


Fig. 13. – UML UnitEvent

4.2.3.3 Événements de changement de shop

Enfin, une troisième catégorie permet de conserver la composition du magasin proposé au joueur après chaque changement, notamment lors d'un reroll. Cette catégorie est également représentée comme une sous-classe d'Event, mais elle se distingue des deux précédentes en ce qu'elle référence non pas une seule unité, mais l'ensemble des unités présentes dans le nouveau magasin.

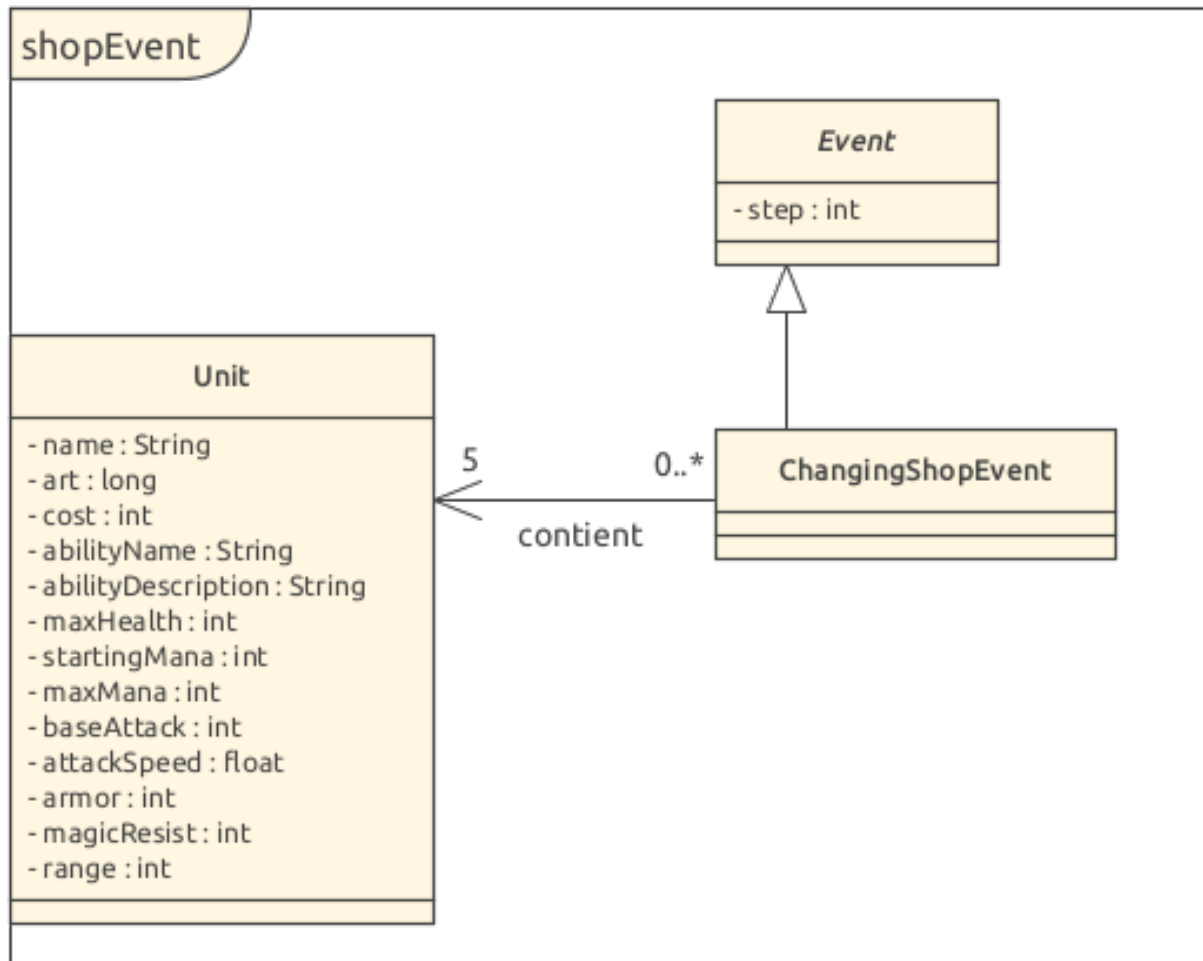


Fig. 14. – UML changingShopEvent

L'ensemble de ces événements et de ces rounds est finalement persisté en base de données, ce qui nous permet, comme expliqué précédemment, à la fois d'assurer la tolérance aux pannes et de proposer un système de replay des parties.

4.3 Architecture réseau

L'un des principaux défis de ce projet est la gestion des connexions, afin de proposer un service à la fois qualitatif et performant.

4.3.1 Authentification

L'authentification a pour objectif d'identifier l'utilisateur, puis d'établir une connexion WebSocket associée à son identifiant (userId), afin de pouvoir le reconnaître tout au long de la session.

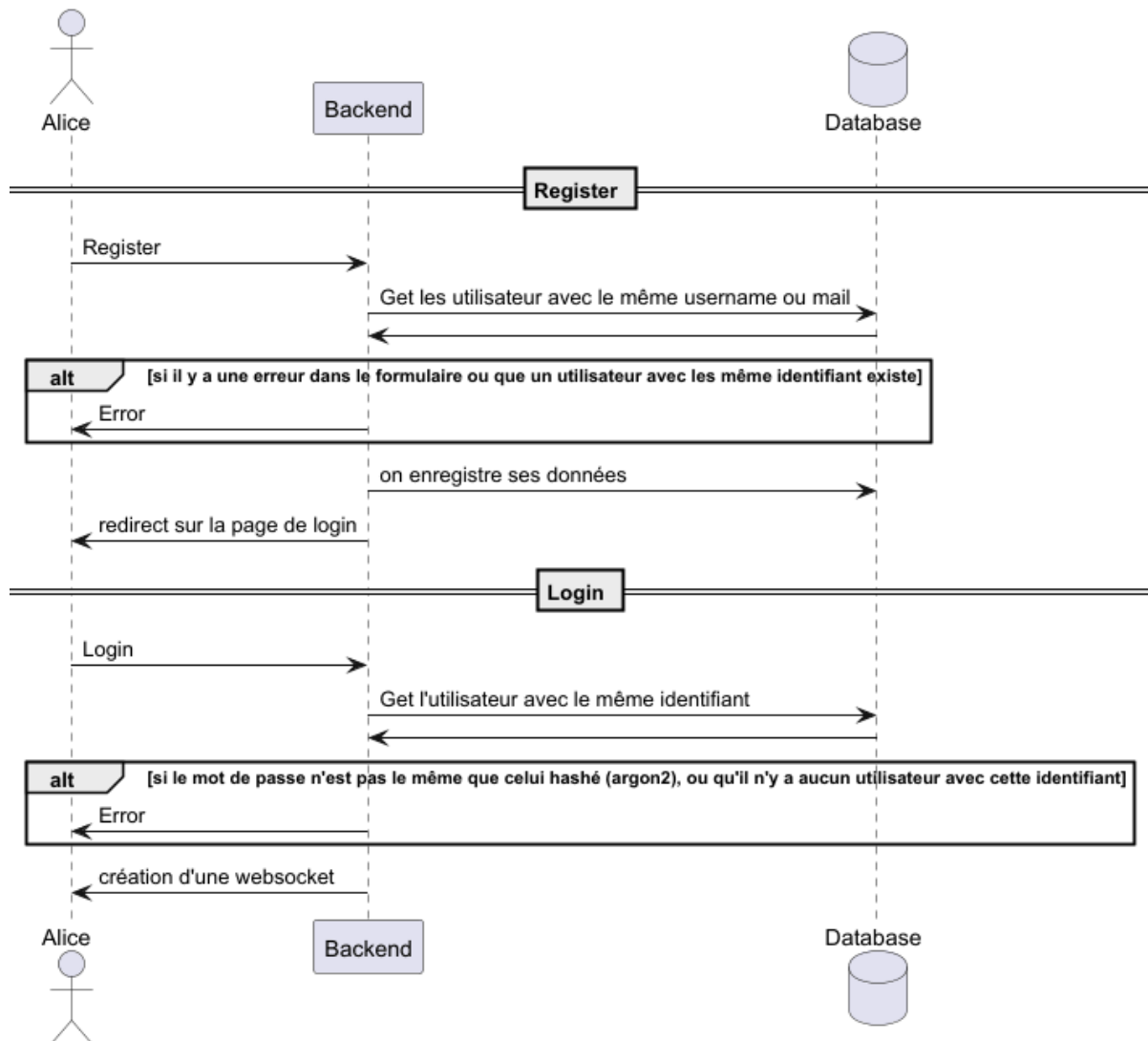


Fig. 15. – Schéma de séquence de l'authentification

Lors de l'inscription, le backend vérifie d'abord auprès de la base de données qu'aucun utilisateur n'existe déjà avec le même nom d'utilisateur ou la même adresse e-mail, et que les données du formulaire sont valides. En cas d'erreur ou de conflit, une erreur est renvoyée au client ; sinon, les données de l'utilisateur sont enregistrées, et celui-ci est redirigé vers la page de connexion.

Lors de la connexion, le backend recherche l'utilisateur correspondant à l'identifiant fourni, puis compare le mot de passe soumis à son équivalent haché à l'aide d'Argon2. Si aucun utilisateur ne correspond ou si le mot de passe est incorrect, une erreur est renvoyée. Dans le cas contraire, une connexion WebSocket est établie avec le client.

4.3.2 Acteurs et moniteur

On doit communiquer de manière bidirectionnelle avec nos joueurs pendant le déroulement des parties. C'est pourquoi on a choisi d'utiliser des WebSockets, implémentées à l'aide de Pekko au sein du framework Play, ce qui amène naturellement à l'utilisation du pattern acteur.

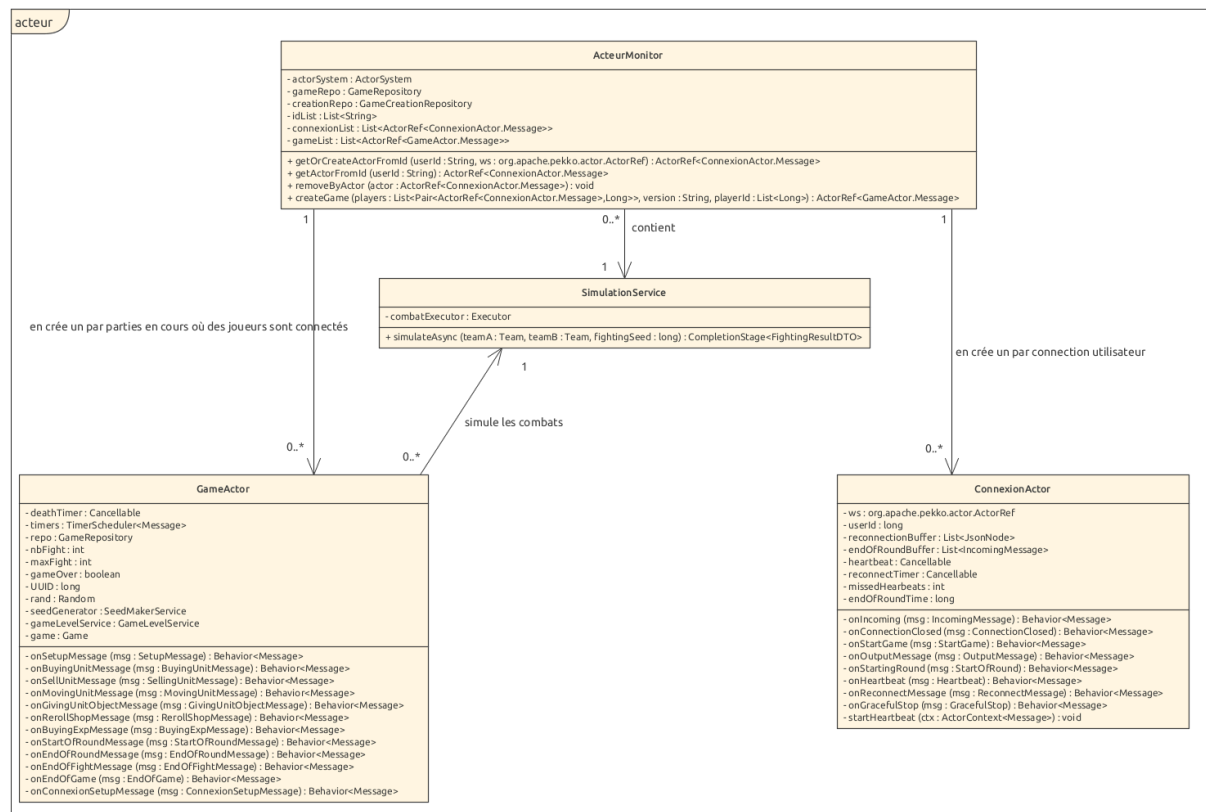


Fig. 16. – UML des acteurs

Nous disposons ainsi d'un **ConnexionActor**, chargé de gérer la connexion d'un joueur, de mettre en tampon (buffer) les messages échangés, et qui s'arrête automatiquement cinq minutes après

une interruption de connexion. Nous disposons également d'un `GameActor`, responsable de la gestion et du déroulement d'une partie. L'utilisation d'acteurs permet de traiter séquentiellement les entrées des différents joueurs par le biais de messages, évitant ainsi les race conditions qui pourraient survenir dans d'autres circonstances.

Afin d'éviter tout problème de concurrence lors de la création ou de la récupération de ces acteurs, on a également mis en place un moniteur, contenant une liste des `ConnexionActor` ainsi qu'une liste des `GameActor`. Ce moniteur est un singleton dont chacune des méthodes touchant à ces listes est synchronisée, ce qui garantit l'absence de toute race condition. Cette solution évite qu'il faille verrouiller l'accès à ces listes à chaque endroit du code où elles seraient utilisées. par exemple si plusieurs instances du `HomeController` tentaient d'accéder simultanément à la même liste de connexions ; à la place, tout accès passe systématiquement par ce moniteur central.

4.3.3 Tolérance aux pannes

On a conçu l'application dans le but d'être aussi résistante que possible aux pannes. Pour cela, comme vu précédemment, l'état actuel de la partie est enregistré à chaque fois qu'un événement est persisté. Ainsi, en cas de panne du serveur, l'état de la partie aura déjà été sauvegardé, et il sera possible de reprendre la partie là où elle s'était arrêtée lors du redémarrage.

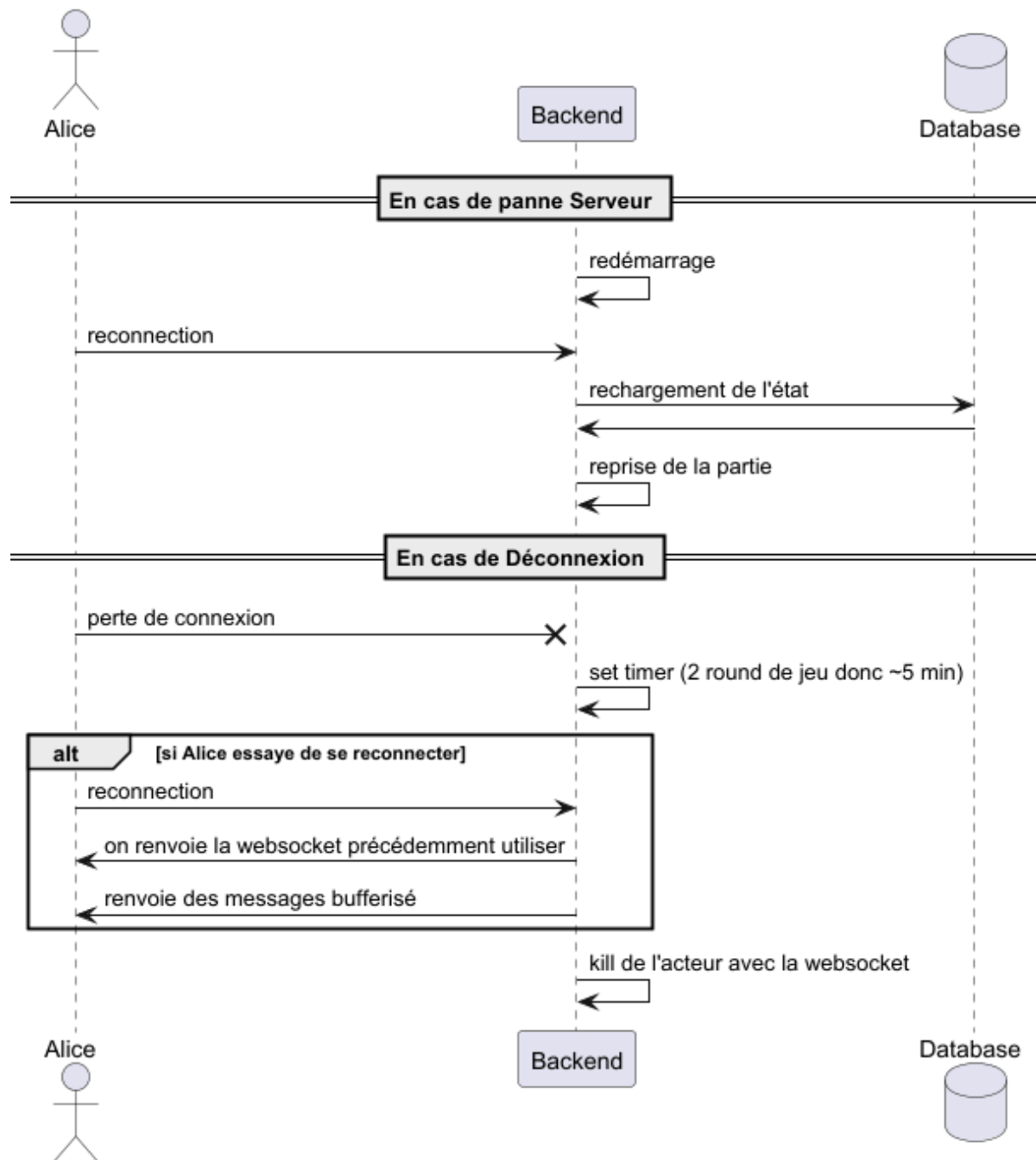


Fig. 17. – Schéma de séquence des pannes

On a également mis en place des mécanismes de reconnexion. Durant les cinq premières minutes suivant la déconnexion d'un utilisateur, son `ConnexionActor` reste actif et met en tampon les différents événements de la partie, afin de pouvoir les lui retransmettre lors de sa reconnexion. Un tampon existe également côté client, afin que tout événement n'ayant pas été confirmé avant la déconnexion soit renvoyé lors de la reconnexion, en vue de sa validation. Si les cinq minutes sont dépassées, l'acteur est arrêté, et le client devra alors se reconnecter et redemander l'établissement d'une nouvelle connexion WebSocket.

Ce délai de cinq minutes représente un compromis entre laisser suffisamment de temps à un joueur pour se reconnecter après un problème réseau temporaire, et éviter de mobiliser inutilement des ressources serveur pour un joueur qui ne reviendrait pas.

4.4 Système de simulation des combats

Pour la simulation des combats, nous nous sommes inspirés d'une architecture couramment utilisée dans le jeu vidéo, qui offre à la fois une bonne modularité et une grande efficacité de traitement : l'Entity Component System (ECS). Le principe de cette architecture n'est pas d'avoir une représentation orientée objet réagissant à des événements, mais de distinguer trois types d'éléments : les entités, qui ne sont qu'un identifiant ; les composants, qui stockent l'ensemble des données propres à une entité ; et les systèmes, qui appliquent la logique métier. Chaque système est exécuté séquentiellement sur l'ensemble des entités, et n'effectue de changement que sur celles possédant les composants requis. Cette architecture est particulièrement adaptée lorsque le nombre d'entités et de systèmes est très élevé.

Dans notre cas, le nombre d'unités en jeu reste restreint : nous avons donc choisi de fusionner les entités et les composants, cette distinction n'apportant pas de bénéfice significatif à notre échelle. Ce qui nous intéresse réellement, ce sont les systèmes et leur exécution. En effet, notre projet prévoyant d'implémenter un grand nombre d'effets différents, il nous fallait une architecture robuste, capable de traiter l'ensemble de ces effets de manière simple et compréhensible. D'où le choix d'associer un système dédié à chaque type de traitement.

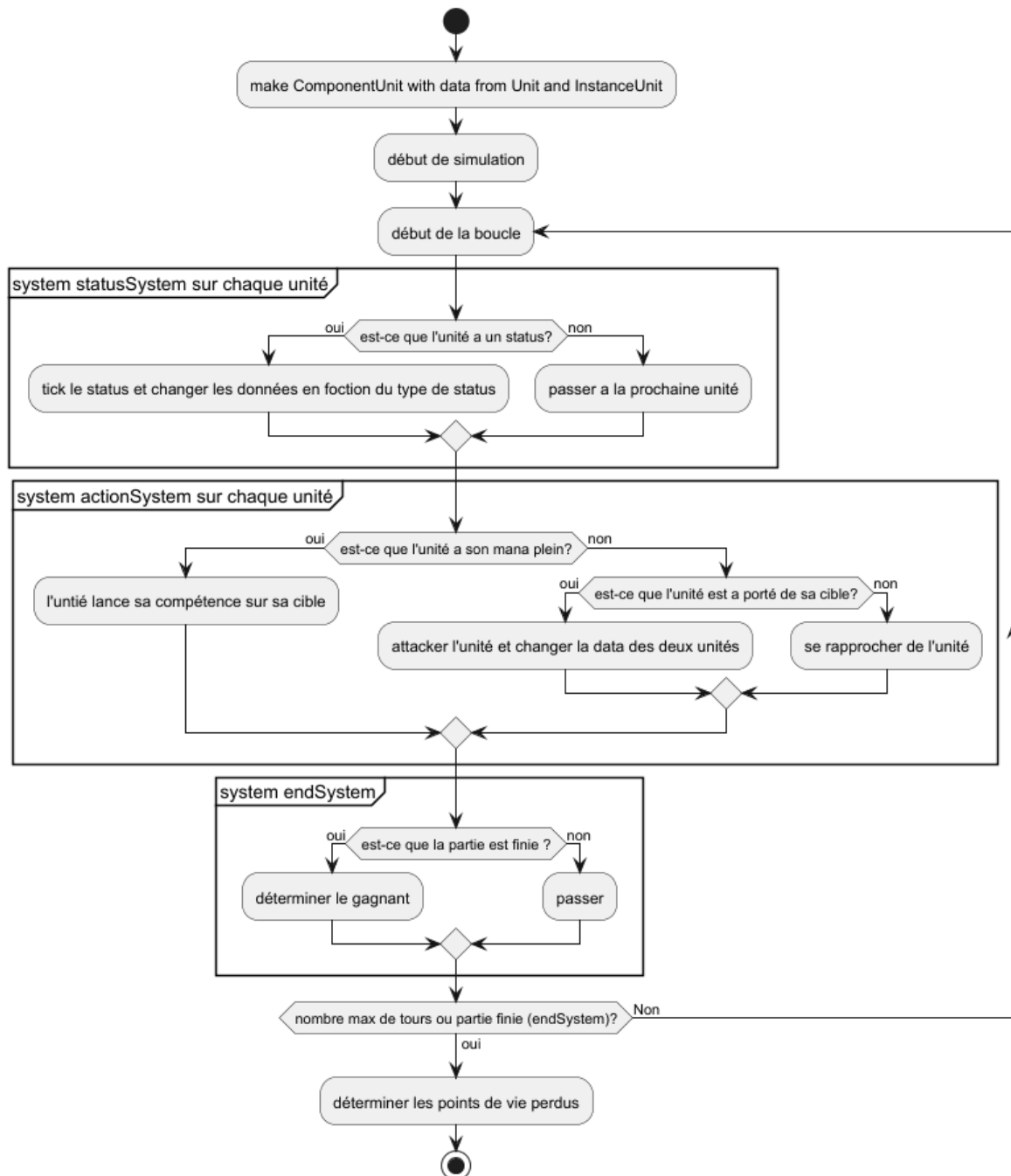


Fig. 18. – schéma d'activité ECS

Concrètement, cela se traduit par un `ComponentUnit`, qui fusionne les données propres à une `InstanceUnit` avec celles de l'unité de base correspondante, afin de disposer de toutes les données nécessaires dès le début du combat. Trois systèmes interviennent ensuite séquentiellement : le `statusSystem`, qui vérifie les statuts (effets appliqués dans la durée) éventuellement infligés aux unités ; l'`actionSystem`, qui déclenche une attaque ou le lancement d'un sort selon le niveau de mana de l'unité ; et enfin l'`endSystem`, qui détermine si le combat est terminé. Notre `FightingService` fait alors itérer ces systèmes en boucle jusqu'à la fin du combat, moment auquel les unités encore en vie sont examinées afin de déterminer les points de vie perdus par chaque équipe.

5 Implémentation

5.1 Configuration du projet

Avant de commencer a coder on s'est d'abord occupez de configurer le projet afin de partir sur des bases de développement propres.

5.1.1 Répo

En premier lieu, on avait le souhait de poser un cadre afin d'instaurer une certaine rigueur dans le travail et les commits. Pour cela on a fait un repository git en s'inspirant de la pratique GitFlow avec ces différentes branche mais les plus importante, une pour les realease et une pour le développement afin de séparer les versions propres des versions qui pourraient encore présenter des défauts.

On a aussi souhaité créer un CI a l'aide des githubs actions afin d'être sur qu'avant chaque merge de pull Request les test ainsi que le coverage soient fait automatique pour éviter des oublies. En ce qui concerne le workflow, on s'est inspiré de ce que propose github pour les application Play et on a rajouté jacoco pour le « test coverage ».

5.1.2 docker

On a aussi créé notre docker compose, qui contrairement à lancer séparément chaque service, docker compose nous permet d'avoir une centralisation des logs ainsi que d'avoir un environ-

nement commun reproductible ou les service peuvent simplement discuter en localhost. Cela permet aussi d'exécuter le projeAuto-battler multijoueur a l'aide d'une seule commande.

5.1.3 application.config

En ce qui concerne la configuration de Play certain de nos service demande une certaine configuration notamment notre Db et Argon2. Tout est présent dans notre fichier de configuration de l'application et il y a aussi un .env pour des données potentiellement plus sensible comme les secrets.

Dans notre application.config on a aussi déclarer deux pool de thread. En effet, on a deux moments ou l'on souhaiterait exécuter des tâches asynchrones sans forcément utiliser les threads principaux. Un premier pool est pour les connexions a la DB qui sont bloquante par nature et comme play est de nature très asynchrone bloquer le thread actuel pour attendre la réponse de la DB paraît insensé. Notre deuxième pool bien que plus discutable est pour la simulation des combats. Même si le système est pour l'instant pas très gourmand en ressource pour le calcul, il va très certainement dans le future prendre plus de temps et de ressource et nous n'avons pas envie que la simulation des combats impacte les performance du reste de l'application. C'est donc pour ca que l'on a ces deux thread pool plutot que de faires ces actions asynchrone sur le main pool.

Par rapport aux tailles des pools nous avons consulté HikariCp qui nous disais d'utiliser deux fois le nombre de coeurs plus un lecteur, mais cela était certainement pour des machines qui ne font pas tourner d'autre services et qui recoivent/produisent beaucoup de requêtes. Notre application a pour but de faire tourner la un docker compose avec toute les images dont on a besoin, notamment la DB et le backend pour l'instant. On a donc décidé d'être économe quant a l'attribution de nos ressources et l'on a arbitrairement donné 8 thread au pool de DB (sachant que le nombre conseillé par HikariCp est 33) et 8 pour notre pool de simulation. Nous sommes conscients du choix d'être plus économe est nous observerons l'évolution du besoin en ressources de connexions afin d'adapter la configuration en conséquence.

5.2 Authentification

On a en premier lieux pour but d'utiliser openIdConnect a la place de mettre en place un système d'authentification afin de déléguer cet aspect de la sécurité qui n'est pas notre point

fort. Cependant notre approche était assez naïve car même si le système marche bien, nous ne nous sommes pas rendu compte lors de nos recherches que cette approche demandait de faire approuver par le vérificateur tiers une page pour demander les informations ainsi que le scope des informations que l'on demanderait. Comme ce genre de démarche peut prendre un certain temps et que l'on souhaitait implémenter un système d'authentification. Nous nous sommes finalement tournés vers un système d'authentification à l'aide des formulaires Play, dans lequel on recueille les informations utilisateur que l'on stocke ensuite dans la base de données.

5.2.1 Hashing

Comme nous avons décidé de nous occuper nous-même de l'authentification il a fallu que l'on trouve un moyen de hasher les mots de passe de nos utilisateurs afin de proposer un minimum de sécurité. Plutôt que de s'essayer à une pratique qui nous est très peu familière, nous avons donc décidé d'opter pour un choix de sécurité et de simplicité, qui est de passer par une bibliothèque de hashing maintenue et sécurisée. Pour cela, on s'est tourné vers un des standards de l'industrie qui est argon2 qui en plus de hasher et de vérifier les mots de passe à travers l'API c'est aussi un algorithme qui prend délibérément du temps et de la mémoire afin de rendre les attaques plus lentes et gourmandes.

5.3 Communication réseau

5.3.1 Format des messages

Les messages échangés entre le client et le serveur sont au format JSON et suivent une structure commune, composée d'un identifiant, d'un type et d'un payload :

```
1 json{
2   "id": "",
3   "type": "",
4   "payload": {
5     "entity1": "",
6     "entity2": ""
```

Json

```
7    }  
8 }
```

Le champ `id` permet d'identifier chaque action de manière unique, notamment pour pouvoir la bufferiser côté serveur et la retransmettre en cas de déconnexion, ou pour l'invalider en cas de triche détectée. Le champ `type` définit l'action concernée (`BuyUnit`, `SellUnit`, `MoveUnit`, `GiveToUnit`, `FuseObject`, `RerollShop`, `BuyExp`), et le contenu du payload varie selon ce type. Par exemple, `MoveUnit` transporte l'identifiant de l'unité déplacée ainsi que sa nouvelle position, tandis que `RerollShop` et `BuyExp` n'ont pas besoin de payload.

Chaque action envoyée par le client fait l'objet d'une validation côté serveur, indépendamment des vérifications déjà effectuées côté frontend, afin de se prémunir contre toute tentative de triche. Par exemple, pour un achat d'unité (`BuyUnit`), le serveur vérifie que l'unité est bien présente dans le shop actuel du joueur, que celui-ci dispose de l'argent nécessaire, et qu'il a la place pour l'accueillir dans son équipe. Le serveur répond ensuite par un message de type `OK` ou `Error`, référencant l'id du message d'origine, ce qui permet au client de savoir précisément quelle action a été acceptée ou rejetée.

5.3.2 Le pont entre WebSocket et acteurs typés : BridgeActor

Play expose les connexions WebSocket sous la forme d'acteurs classiques (`AbstractActor`), alors que le reste de notre logique métier repose sur des acteurs typés Pekko (`AbstractBehavior`), plus sûrs à l'utilisation puisqu'ils forcent un typage strict des messages échangés. Pour faire cohabiter les deux, nous avons introduit un `BridgeActor`, dont le seul rôle est de faire la traduction entre les deux acteurs : chaque message JSON reçu sur la WebSocket est transmis tel quel au `ConnexionActor` typé correspondant sous la forme d'un `IncomingMessage`, et à l'inverse, la fermeture de la connexion (`postStop`) est traduite en un message `ConnectionClosed` envoyé au `ConnexionActor`. Cette séparation permet de garder toute la logique de gestion des connexions et des parties dans des acteurs typés, sans que le typage de Pekko ne soit contraint par l'API WebSocket de Play.

5.3.3 Le ConnexionActor

Chaque joueur connecté est représenté côté serveur par un `ConnexionActor`, qui a pour rôle de router les messages entrants vers la partie en cours (`GameActor`), de transmettre les messages sortants au client, et de gérer le cycle de vie de la connexion (heartbeat, déconnexion, reconnexion).

Routage des messages entrants. À la réception d'un `IncomingMessage`, le `ConnexionActor` lit le champ `type` du JSON et traduit chaque action utilisateur en un message typé correspondant, transmis au `GameActor` de la partie en cours (par exemple, `BuyUnit` devient un `GameActor.BuyingUnitMessage`). C'est le `GameActor`, et non le `ConnexionActor`, qui porte la responsabilité de valider ces actions au regard de l'état de la partie.

Heartbeat et détection de déconnexion. Un mécanisme de heartbeat envoie un message `PING` au client toutes les 30 secondes. Si le client ne répond pas par un `PONG` après trois envois consécutifs, la connexion est considérée comme interrompue et le processus de déconnexion est déclenché. Ce mécanisme permet de détecter une perte de connexion silencieuse (par exemple une coupure réseau côté client qui ne fermerait pas proprement la `WebSocket`).

Buffer de reconnexion. Lorsqu'une connexion est fermée, le `ConnexionActor` n'est pas immédiatement détruit : il conserve chaque message sortant dans un buffer (`reconnectionBuffer`), et démarre un minuteur de 5 minutes. Si le client se reconnecte dans ce délai (`ReconnectMessage`), le timer est annulé et l'acteur reprend son fonctionnement normal ; les messages accumulés dans le buffer peuvent alors être retransmis. Si les 5 minutes s'écoulent sans reconnexion, l'acteur s'arrête (`GracefulStop`) et se désenregistre du moniteur.

Confirmation de réception (ACK). Chaque message envoyé au client est conservé dans le `reconnectionBuffer` jusqu'à ce qu'un message `ACK` portant le même id soit reçu, ce qui permet de le retirer du buffer. Ce mécanisme garantit qu'un message non confirmé sera bien retransmis en cas de reconnexion, plutôt que d'être perdu silencieusement.

Buffer de fin de round. Un second buffer (`endOfRoundBuffer`) conserve les messages reçus après l'heure de fin du round en cours (`endOfRoundTime`). À l'arrivée du round suivant, ces messages en attente sont réinjectés avec un horodatage mis à jour, ce qui évite qu'une action envoyée juste avant la bascule entre deux phases ne soit perdue ou traitée dans le mauvais contexte temporel.

5.3.4 Le GameActor

Le GameActor représente une partie en cours et centralise toute la logique de validation des actions, la diffusion des changements aux autres joueurs, ainsi que le déroulement des rounds et des combats. Validation des actions. Pour chaque action reçue (achat, vente, déplacement d'unité, etc.), le GameActor vérifie l'état actuel de l'équipe concernée avant de l'appliquer. Par exemple, `onBuyingUnitMessage` vérifie que l'équipe existe, que l'unité demandée est valide, et que l'achat est possible, avant de répondre au joueur et de propager l'information aux autres participants de la partie via `tellOtherUsers`.

Diffusion aux autres joueurs. Lorsqu'une action est validée, elle n'est pas seulement confirmée à son auteur : elle est également transmise à tous les autres joueurs de la partie sous forme de `ChangesFromOtherUser`, ce qui permet à chaque client de maintenir une vue à jour de l'état des équipes adverses.

Déroulement des rounds. Le passage d'un round à l'autre est piloté par un minuteur Pekko (`timers.startSingleTimer`), déclenché au début de chaque round pour une durée fixe (`ROUND-TIMEMS`), à laquelle s'ajoute une marge (`INPUTBUFFER`) pour absorber les derniers messages en transit. À l'expiration du timer, le GameActor détermine les combats à simuler pour le round en cours en appariant aléatoirement les équipes encore en vie, à l'aide d'une seed dérivée de la seed globale de la partie (garantissant la reproductibilité de chaque combat).

Simulation asynchrone des combats. Chaque combat est délégué au `SimulationService` de façon asynchrone (`CompletionStage`), ce qui permet au GameActor de continuer à traiter d'autres messages pendant que la simulation s'exécute sur son propre pool de threads dédié. Une fois le résultat disponible, il est transmis directement aux `ConnexionActor` des deux joueurs concernés, puis le GameActor en est informé via un `EndOfFightMessage` pour mettre à jour l'état des équipes (points de vie perdus, victoire/défaite) et vérifier les conditions de fin de partie.

5.3.5 Le moniteur : accès centralisé aux acteurs

Comme évoqué dans la partie conception, l'`ActeurMonitor` centralise l'accès aux `ConnexionActor` et aux `GameActor` actifs, sous la forme de deux listes parallèles synchronisées. Toute méthode qui crée, retrouve ou supprime un acteur (`getOrCreateActorFromId`, `getActorFromId`, `removeByActor`, `createGame`) est déclarée `synchronized`, ce qui garantit qu'un seul thread à la fois peut modifier ces listes et évite ainsi toute race condition en cas d'accès concurrent. Par

exemple si deux requêtes HTTP tentaient simultanément de récupérer ou créer l'acteur d'un même utilisateur.

5.4 Base de données

5.4.1 Les triggers

Comme évoqué dans la partie conception, plusieurs de nos entités (Effect, Event, Strategie, etc.) sont représentées en programmation orientée objet sous forme de hiérarchies de classes. Par exemple, un Effect est soit un ScalingEffect, soit un StatChangingEffect. Une base de données relationnelle ne connaît cependant pas nativement la notion d'héritage : nous avons donc traduit chaque hiérarchie en plusieurs tables liées par clé étrangère, une table par niveau de la hiérarchie (par exemple effect, puis scaling_effect et stat_changing_effect, chacune référençant effect par son id).

Le problème de cette traduction est qu'elle ne garantit, à elle seule, aucune cohérence : rien n'empêche, par exemple, d'insérer une ligne dans effect sans jamais créer la ligne fille correspondante dans scaling_effect ou stat_changing_effect, ce qui laisserait un effet « orphelin », sans comportement concret associé. De la même manière, nos règles métier (une unité ne peut porter plus de 3 objets, une équipe ne peut proposer plus de 5 unités dans son shop, une partie doit comporter exactement 5 pools) ne sont pas non plus exprimables par de simples contraintes SQL classiques, puisqu'elles portent sur un comptage de lignes liées plutôt que sur la valeur d'une seule colonne.

Pour garantir ces règles au niveau de la base elle-même, plutôt que de faire reposer cette responsabilité uniquement sur le backend, ce qui laisserait la porte ouverte à des incohérences en cas d'accès concurrent ou d'oubli, nous avons utilisé des triggers (déclencheurs), sous la forme de CONSTRAINT TRIGGER PostgreSQL. Chaque trigger exécute une fonction plpgsql qui vérifie une règle précise après une insertion, une mise à jour ou une suppression, et lève une exception si la règle est violée, ce qui annule automatiquement la transaction en cours.

Un point important de notre implémentation est que ces triggers sont déclarés DEFERRABLE INITIALLY DEFERRED : la vérification n'est pas effectuée immédiatement après chaque instruction, mais reportée à la fin de la transaction. Ce choix est nécessaire car nos vérifications d'héritage ou de comptage supposent souvent plusieurs insertions liées (par exemple, insérer la ligne event, puis sa ligne fille buy_unit_event) : si la contrainte était vérifiée immédiatement après la première insertion, elle échouerait systématiquement puisque la ligne fille n'existerait

pas encore. En différant la vérification à la fin de la transaction, on s'assure que toutes les insertions liées ont eu le temps de se produire avant que la cohérence ne soit contrôlée.

5.4.2 Deux catégories de règles vérifiées

Vérification de l'héritage. Pour chaque hiérarchie (`event/unit_event`, `effect/scaling_effect`, etc.), une fonction dédiée compte, pour la ligne concernée, le nombre de lignes correspondantes existant dans chacune des tables filles possibles, et vérifie que ce total vaut exactement 1, ni 0 (ligne orpheline, sans sous-type), ni plus de 1 (ligne appartenant à plusieurs sous-types à la fois, ce qui n'aurait pas de sens métier). Par exemple, `check_event_inheritance` s'assure qu'un event correspond à exactement une ligne parmi `leveling_event`, `losing_health_event`, `changing_gold_event`, `changing_shop_event` ou `unit_event`.

Vérification de règles de comptage métier. D'autres triggers portent sur des règles propres au jeu plutôt qu'à la structure d'héritage : `check_unit_objects` empêche une unité de porter plus de 3 objets, `check_team_shop_units` limite le shop d'une équipe à 5 unités maximum, et `check_game_pool/check_game_team` garantissent qu'une partie est toujours associée au bon nombre de pools et d'équipes. Ces triggers s'exécutent après insertion ou suppression sur les tables de liaison concernées (`instance_units_object`, `teams_shop`, `pool`, `team`), ce qui permet de rejeter toute transaction qui laisserait la base dans un état métier invalide, indépendamment de la couche applicative.

5.4.3 Création de la partie : un repository dédié pour une transaction unique

Le lancement d'une partie nécessite de rassembler plusieurs types de données provenant de la base : l'ensemble des unités du jeu (pour constituer les pools de champions disponibles), ainsi que les comptes des joueurs participants (pour créer leurs équipes respectives). Comme évoqué dans la partie conception, ces données doivent être récupérées et assemblées en une seule transaction cohérente, afin d'éviter qu'une partie ne soit créée dans un état partiel ou incohérent. Par exemple si une erreur survenait après la création des équipes mais avant celle des pools.

Pour cela, nous avons isolé toute la logique de création dans un repository dédié, le `GameCreationRepository`, dont l'unique responsabilité est de construire une Game complète et prête à être jouée. Cette création se déroule en plusieurs étapes, exécutées au sein d'une même transaction :

- Récupération des unités. Toutes les unités définies dans le jeu sont chargées depuis la base, afin de pouvoir constituer les pools de champions disponibles pour la partie.
- Création des équipes. Pour chaque identifiant de joueur fourni, l'utilisateur correspondant est recherché en base. Une Team est alors créée pour chaque utilisateur, avec l'or de départ défini par la configuration du jeu (STARTINGGOLD).
- Constitution des pools. Un Pool est créé pour chaque rareté d'unité existante (Rarity), puis chaque unité chargée à l'étape 1 est répartie dans le pool correspondant à sa rareté, sous la forme d'une PoolEntry initialisée avec le nombre d'exemplaires de départ (UNITSTARTINGPOOL).

Une fois ces trois étapes réalisées, la Game, désormais associée à ses équipes et à ses pools, est persistée en une seule fois (`em.persist(game)`), la cascade de persistance de JPA se chargeant de sauvegarder également les entités liées (équipes, pools, entrées de pool).

Isoler cette logique dans un repository spécifique plutôt que de la répartir entre plusieurs services présente deux avantages. D'une part, cela garantit que la création d'une partie reste atomique : toute la construction se fait dans le contexte d'une seule transaction JPA, si bien qu'une erreur à n'importe quelle étape (par exemple un utilisateur introuvable) annule l'ensemble de la création plutôt que de laisser une partie à moitié initialisée en base.

D'autre part, cela centralise en un seul endroit toute la connaissance de « ce qu'il faut pour démarrer une partie », ce qui facilite l'évolution de cette logique si de nouveaux éléments devaient être initialisés au lancement d'une partie (par exemple des objets ou des règles spécifiques à un mode de jeu).

La création d'une partie n'a pas été implémenté dans le `gameRepository` car on a estimé que c'est un comportement qui n'a pas lieux d'être et ne doit être utilisé qu'à un seul endroit, d'où son isolement.

5.5 Algorithmes

5.5.1 Explication d'A* pour le pathfinding

Lors de la simulation des combats, chaque unité doit soit attaquer, soit se déplacer jusqu'à se trouver à portée d'une cible. Il nous fallait donc un algorithme de pathfinding capable de calculer le chemin le plus court entre une unité et sa cible. Afin de ne pas frustrer le joueur en lui montrant une unité emprunter un chemin visiblement non optimal, nous avons choisi

l'algorithme A^* , un standard largement utilisé pour ce type de problème, notamment dans le jeu vidéo.

5.5.1.1 Rappel du fonctionnement d' A^*

A^* est un algorithme de recherche de chemin qui explore un graphe (ici, notre grille de cases) en cherchant à atteindre une case d'arrivée depuis une case de départ, tout en minimisant le coût total du trajet. Sa particularité, par rapport à un algorithme de parcours plus naïf comme un simple parcours en largeur (BFS), est qu'il priorise intelligemment les cases à explorer plutôt que de les explorer toutes indifféremment. Pour cela, A^* attribue à chaque case explorée un score f , calculé comme la somme de deux valeurs :

g : le coût réel du chemin déjà parcouru depuis le départ pour atteindre cette case (dans notre cas, simplement le nombre de cases traversées, chaque déplacement coûtant 1). h : une estimation du coût restant pour atteindre l'arrivée depuis cette case, appelée heuristique. Cette estimation doit être admissible, c'est-à-dire qu'elle ne doit jamais surestimer le coût réel restant, sous peine de produire un chemin qui ne soit plus garanti optimal.

À chaque étape, l'algorithme retire de la liste des cases à explorer (open list) celle ayant le score f le plus bas, et explore ses voisines. Une fois qu'une case a été explorée, elle est marquée comme définitivement traitée (closed list) et ne sera plus reconsidérée, sauf si un chemin strictement plus court vers elle est découvert par la suite. L'algorithme s'arrête dès que la case d'arrivée est retirée de la open list, garantissant qu'aucun chemin plus court n'existe. Le chemin final est ensuite reconstruit en remontant, de case en case, le « parent » de chaque case jusqu'au point de départ.

C'est précisément cette combinaison entre coût réel déjà parcouru (g) et estimation du coût restant (h) qui rend A^* à la fois complet (il trouve toujours un chemin s'il en existe un) et efficace (il évite d'explorer inutilement des cases qui s'éloignent manifestement de la destination), contrairement à un algorithme comme Dijkstra qui, en l'absence d'heuristique, explore le graphe de façon plus large et donc moins ciblée.

5.5.1.2 Représentation de la grille hexagonale

Notre terrain de combat est hexagonal, ce qui complique légèrement l'application d' A^* , pensé à l'origine pour des grilles carrées. Nous avons choisi de représenter chaque case selon un

système de coordonnées cubiques (x, y, z , avec la contrainte $x + y + z = 0$), une représentation classique pour les grilles hexagonales qui simplifie grandement le calcul des voisins d'une case (six directions fixes) ainsi que la distance entre deux cases, calculée simplement comme le maximum des différences absolues sur chacun des trois axes .

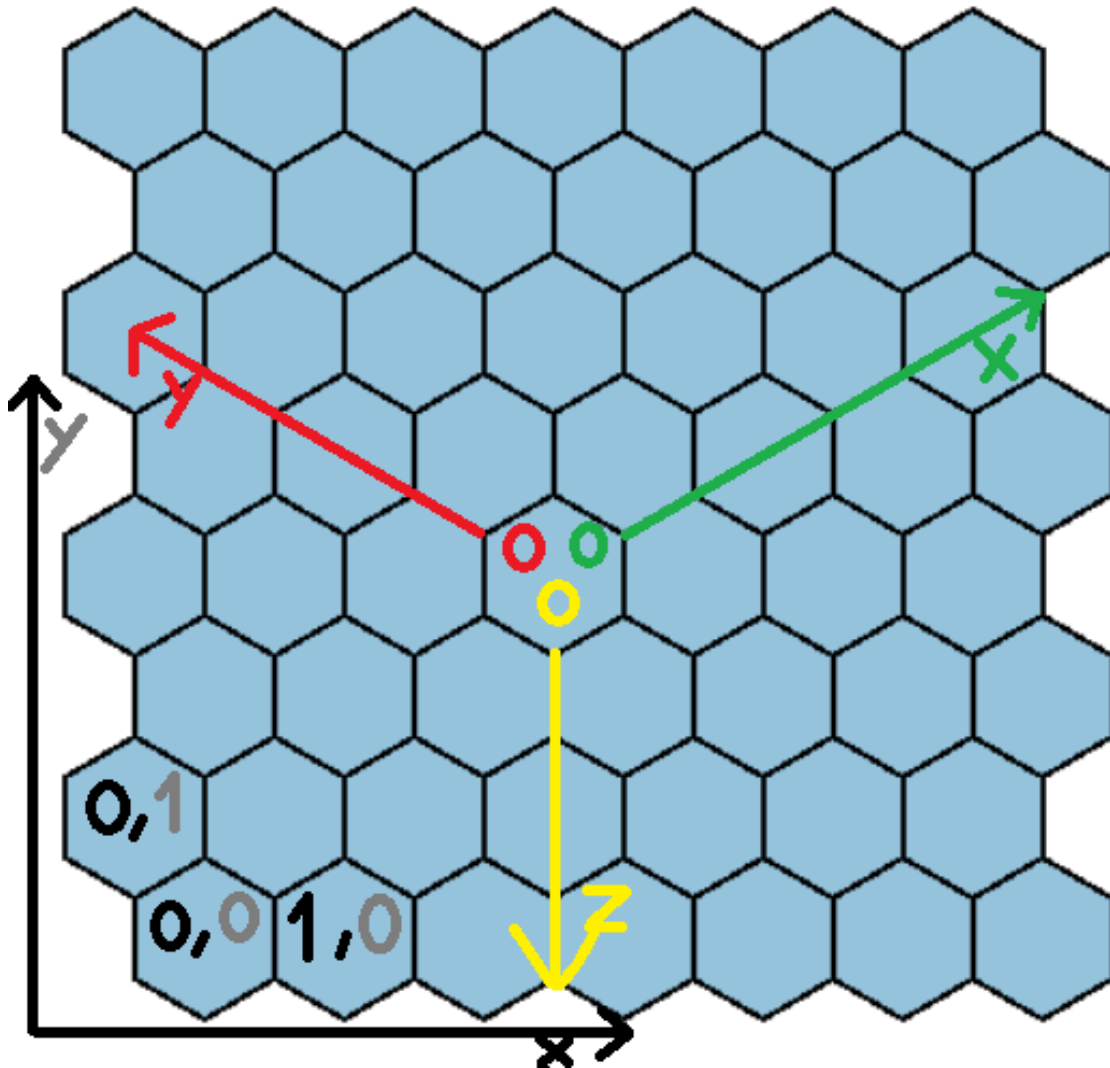


Fig. 19. – schéma explicatif des représentation cubique et classique

Cette représentation cubique coexiste avec une représentation en coordonnées « offset » (col, row), plus intuitive pour l'affichage 2D côté frontend et pour l'indexation de notre grille en mémoire (un simple tableau à deux dimensions `Tile[][]`). Un jeu de fonctions de conversion (`offsetToCube` et `cubeToOffset`) permet de passer de l'une à l'autre selon le besoin.

5.5.1.3 Notre implémentation

Chaque case de la grille (Tile) porte directement l'état nécessaire à la recherche A (g, h, parent, état de recherche), plutôt que de passer par une structure de données externe (comme une Map). Ce choix simplifie l'implémentation, mais impose une précaution : puisque l'état est stocké sur les cases elles-mêmes, une recherche pourrait « polluer » l'état laissé par la précédente. Pour l'éviter, chaque appel à `findPath` conserve la liste des cases qu'il a effectivement modifiées et les réinitialise systématiquement à la fin de son exécution, plutôt que de devoir parcourir l'intégralité de la grille à chaque fois. Un compromis qui garde de bonnes performances tout en assurant que chaque recherche reparte d'un état propre.

L'heuristique h utilisée est la distance hexagonale définie plus haut, qui est admissible puisqu'elle correspond exactement au nombre minimal de déplacements nécessaires en l'absence d'obstacle. Elle ne peut donc jamais surestimer le coût réel restant.

5.5.2 Service de Matchmaking

Le matchmaking doit maintenir une liste des joueurs en attente ainsi que leur connexion associée, afin de pouvoir démarrer une partie une fois qu'un nombre suffisant de joueurs est atteint.

Les contrôleurs de Play sont par nature concurrents, et c'est déjà un problème que nous avons mentionné dans la section concernant l'**ActeurMonitor**.

Nous avons donc opté pour un `MatchmakingService` singleton, dans lequel les méthodes d'ajout d'un joueur (`addPlayer`) et de tentative de création de partie (`tryCreateGame`) sont déclarées synchronisées.

Ce choix garantit qu'un seul thread à la fois peut exécuter l'ensemble de la séquence « ajouter un joueur à la file d'attente, puis vérifier si le seuil de 8 joueurs est atteint et créer la partie le cas échéant ». La vérification du seuil et la création de la partie se font donc de manière atomique du point de vue des autres contrôleurs, ce qui élimine tout risque de créer deux parties simultanément ou d'assigner un même joueur à deux parties différentes.

Ce même principe de synchronisation centralisée est également celui déjà utilisé dans l'`ActeurMonitor` pour la gestion des connexions actives. Le `MatchmakingService` s'appuie d'ailleurs directement sur ce dernier (`monitor.getActorFromId`, `monitor.createGame`) pour

retrouver la connexion d'un joueur et déléguer la création effective de la partie une fois le quota atteint.

5.5.3 Service de SeedMaker

Comme expliqué précédemment, notre jeu repose sur de nombreux mécanismes aléatoires, mais on souhaite pouvoir proposer un système de replay des parties. Pour que ce dernier soit fiable, il est nécessaire de stocker les graines (seeds) utilisées, afin que chaque décision aléatoire, une fois rejouée, mène exactement aux mêmes conséquences.

5.5.3.1 Le problème de l'ordre des événements

La principale difficulté rencontrée dans notre conceptualisation initiale provient de la façon dont nous ordonnons les événements. Chaque événement possède un step qui définit son ordre de passage, mais ce step est propre à chaque équipe : rien ne permet donc de connaître l'ordre relatif entre deux événements d'un même round appartenant à des équipes différentes.

Ce manque d'ordre global pose un problème concret pour tout événement dépendant de l'aléatoire et calculé côté serveur. Le reroll du shop, par exemple, est calculé par le GameActor, qui détermine alors quels champions seront proposés à partir de la seed. Si l'ordre entre deux rerolls d'équipes différentes n'est pas connu lors du replay, la seed risque d'être consommée dans un ordre différent de celui de la partie originale, ce qui produirait des résultats différents et romprait le déterminisme recherché.

5.5.3.2 Solutions envisagées

Une première option aurait été d'introduire un step global, commun à toutes les équipes plutôt que propre à chacune. Cette solution aurait cependant nécessité que chaque round ait connaissance du step global courant, ce qui implique soit une synchronisation de ce step entre toutes les équipes, soit sa prise en charge centralisée par la Game elle-même. Dans les deux cas, une complexité supplémentaire non négligeable dans la gestion de l'état de la partie.

On a donc préféré résoudre le problème autrement : plutôt que d'imposer un ordre global entre les équipes, nous avons choisi de dériver une seed distincte par équipe à partir de la seed globale

de la partie, de façon déterministe. Ainsi, tout événement aléatoire propre à une équipe dépend uniquement de la seed de cette équipe, elle-même reproductible à l'identique à chaque replay. L'ordre entre les événements de deux équipes différentes n'a alors plus d'importance, puisque leurs séquences aléatoires respectives sont totalement indépendantes l'une de l'autre.

Ce même problème se posait pour la simulation des combats, ceux-ci reposant également sur de l'aléatoire (notamment pour les coups critiques). Nous avons appliqué la même solution : une seed dédiée est dérivée de la seed de la partie pour chaque combat, ce qui garantit sa reproductibilité indépendamment de l'ordre dans lequel les combats sont traités.

5.5.3.3 Génération des seeds dérivées

Pour que les seeds dérivées soient suffisamment différentes les unes des autres et ne présentent pas de motif prévisible, chaque seed est générée à partir d'un contexte: une chaîne de caractères décrivant brièvement son usage (par exemple l'identifiant de l'équipe ou du combat concerné), qui est ensuite hashé de façon déterministe pour produire la seed finale.

On n'a pas voulu réutiliser notre service de hashing basé sur Argon2 pour cet usage : cet algorithme est volontairement conçu pour être lent et gourmand en ressources (afin de ralentir les attaques par force brute sur les mots de passe), ce qui le rend inadapté à une génération de seed devant rester rapide, et il nécessite en outre un sel non réutilisable tel quel dans un contexte déterministe. Nous nous sommes donc tournés vers SHA-256, une fonction de hash cryptographique rapide et surtout parfaitement déterministe : pour une même entrée, elle produit toujours exactement la même sortie, ce qui est précisément la propriété recherchée pour garantir la reproductibilité de nos seeds lors d'un replay.

5.5.4 Test

La validation du système repose sur une suite de tests unitaires écrite avec JUnit 5, complétée par Mockito pour l'isolation des dépendances et par le module ActorTestKit de Pekko pour les composants concurrents. Les structures de données du module de pathfinding (HexPosition, Tile, HexGrid) et l'algorithme A^* qui les exploite ont été testés de manière déterministe : les tuiles sont reliées manuellement en dehors de toute grille réelle, ce qui permet de contrôler précisément l'ordre de traitement de la file de priorité et de couvrir des cas limites difficiles à provoquer autrement, tels que la mise à jour d'un nœud encore ouvert lorsqu'un chemin moins coûteux est découvert, ou l'ignorance d'un nœud déjà fermé. Cette démarche a par ailleurs permis d'identifier une erreur de bornes dans la construction de la grille hexagonale, corrigée avant l'écriture des tests correspondants. Les composants asynchrones, le gestionnaire de connexions (ActeurMonitor) et les acteurs typés (ConnexionActor, GameActor), ont fait l'objet

d'une stratégie différente : les dépendances externes (persistance, services métier, système d'acteurs) sont simulées via Mockito, tandis qu'un système d'acteurs de test réel est instancié pour observer le comportement effectif des échanges de messages, notamment la gestion du heartbeat, le tampon de reconnexion et la propagation des actions de jeu entre les acteurs. Cette approche privilégie la vérification du comportement observable plutôt que de l'état interne, difficilement accessible sur des acteurs encapsulant fortement leurs données.

La couche service métier (GameLevelService, HashService, MatchmakingService, SeedMakerService, SimulationService) a fait l'objet d'une isolation systématique de ses dépendances externes : JPAApi et Config sont simulés via Mockito pour contrôler le chargement des données de progression, tandis que le service de hachage est validé par exécution réelle de l'algorithme Argon2id. Avec des paramètres réduits pour la performance des tests, afin de garantir que le hachage, la vérification et l'effacement des mots de passe en mémoire se comportent conformément aux spécifications, y compris en cas d'échec. Le service de génération de graines a fait l'objet d'une attention particulière : outre la vérification du déterminisme des graines dérivées, un mock statique de MessageDigest a permis de couvrir la branche défensive normalement inatteignable correspondant à l'absence de l'algorithme SHA-256 sur la JVM. Le service de matchmaking a été validé par simulation de l'arrivée successive de joueurs, vérifiant le déclenchement de la création de partie au seuil attendu ainsi que la réinitialisation correcte de la file d'attente. Le service de simulation asynchrone, enfin, a été isolé de son moteur de combat interne par interception de sa construction (mockConstruction), ce qui a permis de vérifier la délégation vers le dispatcher dédié sans dépendre du calcul réel du combat.

Le moteur de simulation de combat lui-même (FightingService, FightingContext, ainsi que les systèmes StatusSystem, EndSystem et ActionSystem) présente la particularité d'instancier directement ses collaborateurs internes, sans injection de dépendances. Pour préserver l'isolation malgré ce couplage fort, chaque construction interne (contexte de combat, systèmes de mise à jour, unités de combat, objets de résultat) a été interceptée via le même mécanisme mockConstruction, permettant de vérifier la logique d'orchestration (bornage de la boucle de simulation, conditions d'arrêt, calcul des points de vie perdus) indépendamment du comportement réel des systèmes sous-jacents. Combinée à la capture des arguments de construction, cette approche a également permis de valider EndSystem sans dépendre des accesseurs internes de ses objets d'événement, révélant au passage un déréférencement non protégé (getFirst()) sur une liste d'unités vivantes potentiellement vide) ainsi qu'un champ non initialisé exposé par getWinner(), susceptible de provoquer une exception d'unboxing. Certaines branches, en particulier celles dépendant de types non exposés dans le périmètre analysé (stratégies de ciblage des capacités actives, entités de persistance liées aux comptes utilisateurs) n'ont pu être couvertes avec la même rigueur et ont été identifiées comme axes d'amélioration de la suite de tests.

Le LoginRepository, en charge de la persistance des comptes utilisateurs, a été validé par des tests d'intégration reposant sur Testcontainers plutôt que sur une isolation par mocks : une véritable instance PostgreSQL est instanciée dans un conteneur Docker éphémère, initialisée par un script SQL dédié, puis injectée dans une application Play complète via GuiceApplica-

tionBuilder. Cette approche permet d'exercer le repository contre un moteur de base de données réel et non contre une simulation de son comportement, garantissant ainsi que le mapping objet-relationnel, les contraintes du schéma et les requêtes effectivement générées se comportent comme attendu en production. Les scénarios couvrent le cycle de vie complet d'un utilisateur : création puis récupération par email et par nom d'utilisateur (avec vérification de la cohérence entre les deux résultats), création suivie d'une relecture directe de l'entité persistée, ainsi que la récupération exhaustive de l'ensemble des comptes enregistrés, y compris la vérification négative qu'un utilisateur non ajouté n'y figure pas. Chaque test prend soin de supprimer les données qu'il a introduites à son issue, afin de préserver l'indépendance des scénarios malgré le partage du même conteneur pour toute la suite. Cette stratégie, plus coûteuse en temps d'exécution qu'une suite unitaire classique, a été réservée à la seule couche de persistance, là où la fidélité au comportement réel du système de gestion de base de données prime sur la rapidité d'exécution.

Avec tous ces tests on obtiens seulement 46% de code coverage et 34% de branche coverage. Il sera donc nécessaire dans le future de tester beaucoup plus surtout les acteur et les entités qui sont les deux grosse partie manquante de notre coverage.

6 Difficultés rencontrées

On va ici parler des difficultés rencontrées lors de ce travail de bachelor. C'est une partie importante, car il s'agit de défis qu'on avait pas anticipés, et dont il est essentiel de tirer un maximum d'enseignements afin de ne plus y être confrontés à l'avenir, mais surtout afin de progresser.

6.1 Première base de données

Abordons tout d'abord la conception et l'implémentation de notre base de données. Lors de la conception, on avait décidé de nous orienter vers une base de données relationnelle. Cependant, parmi les cours que j'ai suivi durant mon cursus ceux parlant de base de donnée avait quelques lacunes quant à l'aspect pratique de la conceptualisation et l'implémentation d'une base de donnée. Je me suis donc retrouvé à devoir rattraper le cours de Base de Données Relationnelles de la HEIG afin de combler mes lacunes sur le sujet.

Ce rattrapage a entraîné un ralentissement, aussi bien au niveau de la conception que de l'implémentation. C'est notamment à cette occasion que j'ai découvert le fonctionnement de l'héritage dans une base de données relationnelle, ainsi que la manière de garantir la cohérence de cet héritage à l'aide de triggers.

6.2 Découverte de Play et notamment Pekko avec le pattern acteur

Le framework utilisé était forcément un peu différent de ce que j'avais pu rencontrer jusque-là, sans présenter toutefois de différence fondamentale, si ce n'est que l'ensemble du système

de communication repose sur Pekko, qui utilise le pattern acteur. Il s'agit d'un pattern que je n'avais jamais rencontré auparavant, et qui m'a demandé un certain temps avant d'en comprendre le fonctionnement, ainsi que la manière d'en tirer pleinement parti dans l'implémentation de mon projet.

6.3 difficulté de focus sur les technologie utile

Lors de la phase initiale et de l'état de l'art, n'ayant encore jamais eu à faire de choix d'architecture ou de technologies dans les différents projets auxquels j'avais participé jusqu'ici (principalement des laboratoires), j'ai perdu beaucoup de temps à m'éparpiller sur des aspects peu utiles, qui étaient soit trop poussés pour le périmètre de mon projet, soit inutiles au vu de sa nature ou du framework choisi.

Je n'ai pas non plus pris le temps d'analyser suffisamment en profondeur les points réellement critiques de mon projet : par exemple, le problème rencontré avec OpenIDConnect, qui nécessitait des démarches administratives avec des tiers, ne s'est révélé à moi qu'au moment de commencer son implémentation, plutôt qu'en amont lors de l'état de l'art.

6.4 Difficulté de conception et de poser les choses à plat

Dans les paragraphes précédents, nous avons discuté des difficultés rencontrées lors de ce travail. Mais je pense que la plus grande difficulté que j'aie rencontrée provient de mon inexpérience et de ma naïveté dans mon approche de ce travail.

En effet, le fait d'avoir moi-même proposé le sujet impliquait que la plupart des décisions de conception et d'architecture découlaient finalement de la façon dont je percevais le projet. Pour rester cohérent avec cette vision, il aurait fallu que je formalise une conceptualisation claire des différentes parties bien en amont. Or, alors même qu'on me l'avait conseillé, mon approche a été de me dire que je verrais les choses au fur et à mesure, persuadé de savoir précisément ce que je voulais obtenir. Cette mentalité n'a cependant mené, tout au long de l'implémentation, qu'à de nombreux allers-retours, car j'avais oublié certains éléments, sans compter les fonctionnalités que j'ai dû sacrifier à cause de cette négligence initiale.

6.5 Confusion entre vision du jeu et besoins logiciels

C'est le plus gros projet sur lequel j'ai eu l'occasion de travailler, et le fait d'en être moi-même le client m'a en réalité desservi : en plus du travail de bachelor à proprement parler, j'avais une véritable réflexion à mener sur ce que je voulais construire, réflexion qui n'a pas été menée suffisamment en amont pour être efficace au moment de l'implémentation.

Je pense que j'abordais le projet en sachant exactement ce que je voulais obtenir en tant que jeu, mais que j'étais en revanche complètement perdu quant aux fonctionnalités nécessaires du point de vue logiciel. Cela m'a empêché de me rendre compte, dès le départ, de l'ampleur du travail supplémentaire que cela allait représenter, et qu'il aurait fallu commencer par me concentrer sur un ensemble de fonctionnalités minimal. Quitte à ce que le jeu ne soit pas encore « amusant », plutôt que de chercher à tout construire en même temps. C'est pourtant la direction qui aurait dû être prise dès le départ.

Bien que cela paraisse évident aujourd'hui, et qu'il faille admettre qu'à l'époque j'en connaissais déjà la théorie, je n'ai, dans la pratique, pas su séparer suffisamment les choses pour obtenir un jeu minimal viable, tout en implémentant les fonctionnalités logicielles nécessaires à son fonctionnement.

7 Conclusion

7.1 Bilan du projet

Ce travail de bachelor avait pour objectif de concevoir et de développer un jeu multijoueur de type auto-battler, accessible via le web, en mettant en œuvre une architecture logicielle complète et professionnelle : interface utilisateur, logique métier, persistance des données et communication client-serveur en temps réel.

Le cahier des charges définissait comme obligatoires un système de replay fonctionnel ainsi qu’une gestion complète du cycle de vie du matchmaking. Ces deux fonctionnalités ont été conçues et en grande partie implémentées côté backend, mais n’ont pas pu être finalisées dans le temps imparti : il manque notamment la partie frontend et les messages acteur nécessaires pour rendre le replay utilisable, ainsi que la gestion du retrait d’un joueur de la file d’attente avant le lancement d’une partie. Cet écart entre le périmètre initialement prévu et le périmètre effectivement livré s’explique principalement par les difficultés de conception évoquées au chapitre précédent, et constitue un axe de travail prioritaire pour la suite du projet.

Sur le plan technique, une architecture client-serveur a été mise en place autour du Play Framework et de Pekko, avec un système d’acteurs permettant de gérer les connexions et le déroulement des parties de manière concurrente, sans race condition. La persistance des données repose sur une base relationnelle PostgreSQL, conçue pour offrir à la fois une tolérance aux pannes et un système de replay basé sur un modèle événementiel normalisé. Le système de combat s’appuie sur une architecture Entity Component System adaptée à nos besoins, combinée à un algorithme A^* pour le déplacement des unités sur une grille hexagonale. Enfin, le frontend repose sur PixiJS pour l’affichage performant d’un grand nombre de sprites, et l’ensemble du projet est accompagné d’un pipeline CI/CD basé sur GitHub Actions et Docker.

7.2 Limitations et perspectives

Plusieurs pistes d'amélioration ont été identifiées au cours du développement, mais n'ont pas pu être traitées dans le cadre de ce travail :

La gestion du départ d'un joueur de la file d'attente de matchmaking avant le lancement d'une partie n'est pas encore implémentée. L'authentification déléguée via OpenID Connect, initialement envisagée, a été abandonnée au profit d'une authentification classique par formulaire et hachage Argon2, en raison des contraintes administratives liées à la certification auprès des fournisseurs tiers ; elle pourrait être réintroduite en complément dans une version future soit en passant par Keycloak soit en faisant la demande au fournisseurs tiers. L'implémentation du système de replay, même si la conception a été faite et qu'il y a une grosse parti de l'implémentation il manque une partie frontend et des messages acteur pour que la fonctionnalité soit utilisable. Enfin, a cause des difficulté rencontrer, le frontend n'est pas entièrement fini et tester notemment la partie de combats.

Ces éléments constituent des axes d'amélioration naturels pour une éventuelle poursuite du projet au-delà de ce travail de bachelor.

7.3 Apprentissages personnels

Au-delà de l'aspect technique, ce projet représente le plus important que j'ai eu l'occasion de mener à ce jour, et il m'a permis d'apprendre énormément, tant sur le plan technique que méthodologique.

Techniquement, j'ai dû combler plusieurs lacunes en cours de route, notamment en bases de données relationnelles, j'ai dû rattraper le cours de Bases de Données Relationnelles de la HEIG en parallèle du projet. Ainsi que sur le pattern acteur avec Pekko, que je découvrais pour la première fois et dont la logique de programmation concurrente m'a demandé un temps d'adaptation certain.

Méthodologiquement, ce travail m'a surtout confronté à l'importance d'une conceptualisation rigoureuse en amont de l'implémentation. Étant moi-même à l'origine du sujet, j'avais une vision du jeu que je voulais construire, mais j'ai sous-estimé l'ampleur de la réflexion logicielle nécessaire pour y parvenir et ma vision des besoins logiciels était floue, ce qui a entraîné de nombreux allers-retours au cours de l'implémentation faute d'avoir posé les choses à plat suffisamment tôt. Si c'était à refaire, je privilégierais une approche par fonctionnalités minimales

(MVP), en utilisant mon temps de l'état de l'art pour faire des recherches plus approfondis sur ce dont j'ai besoin, et aussi poser très tôt tous mes fonctionnalités pour gagner du temps plus tard et voir les problèmes plus tôt.

Ce projet m'a ainsi permis non seulement de renforcer mes compétences techniques, mais surtout de mieux comprendre ma propre manière de travailler sur un projet de cette envergure, un enseignement que je compte mettre à profit dans mes futurs projets, notamment professionnels dans le domaine du jeu vidéo.

8 Journal de travail

Date	Semaine	Description	Matiné. [h]	Après-midi. [h]
<i>Le journal de travail continue à la page suivante.</i>				
20.02.2026	1	Réalisation du cahier des charges et première réunion	3	3
26.02.2026	2	Mise au propre du cahier des charges	0	2
27.02.2026	2	fin du planning et établissement des besoins	3	4
06.03.2026	3	recherche de la résolutions des problèmes amené par les besoins	4	3
20.03.2026	5	recherche pour la réalisation de l'état de l'art	3	4
22.03.2026	6	planification pour l'état de l'art	0	3
27.03.2026	6	recherche sur le frontend ainsi que début de rapport	4	4
30.03.2026	7	rédaction du rapport	4	3
09.04.2026	8	recherche sur les base de donnée	0	3
13.04.2026	9	finalisation de l'état de l'art et de cette parti du rapport	3	5
17.04.2026	9	début de la conceptualisation de la base de donnée	4	3
20.04.2026	10	conception uml pour le système unité	5	0
23.04.2026	10	refonte du schéma de base de donnée	0	3
24.04.2026	10	finalisation d'un des deux schéma entité associations	4	2

Date	Semaine	Description	Matiné. [h]	Après-midi. [h]
27.04.2026	11	réalisation du deuxième schéma entité et rapport	4	5
03.05.2026	12	schéma de base de données avec et sans replay	2	4
04.05.2026	12	travail sur le schéma de classe pour la simulation	3	4
05.05.2026	12	travail sur le schéma de classe pour la simulation	0	3
08.05.2026	12	itération sur le schéma de base de donnée pour avoir un replay	3	4
11.05.2026	13	travail sur le schéma de classe	3	0
15.05.2026	13	travail sur la refont du schéma EA pour la BD, et travail sur le schéma de classe	3	5
17.05.2026	14	passage du rapport sur typst	0	5
18.05.2026	14	écriture du rapport intermédiaire et recherche sur les acteur	3	3
19.05.2026	14	finalisation du rapport intermédiaire	4	0
20.05.2026	14	modification du cahier des charges	1	0
22.05.2026	14	recherche sur le pipeline CI/CD et github actions	3	2
25.05.2026	15	implémentation de placeholder pour le frontend	4	4
29.05.2026	15	ajout de fonctionnalité au frontend	4	6
01.06.2026	16	préparation de la présentation	4	4
16.06.2026	18	implémentation du système de login	3	5
17.06.2026	18	implémentation de docker compose et de la DB	4	2
18.06.2026	18	implémentation de la base de donnée	4	4
19.06.2026	18	finalisation de la DB login et des repo	4	3
22.06.2026	19	test du repository	2	4
24.06.2026	19	fin des test login et login fonctionnel	3	5
25.06.2026	19	finalisation du login, mise au propre	3	3

Date	Semaine	Description	Matiné. [h]	Après-midi. [h]
26.06.2026	19	recherche acteur pekko	3	6
29.06.2026	20	refocus sur l'essentiel et reduction du scope	3	5
30.06.2026	20	travail sur les acteurs	4	5
01.07.2026	20	websocket après login et schéma OO	3	5
02.07.2026	20	continuation de l'implémentation de la DB et recherche sur les trigger	3	5
03.07.2026	20	travail sur les acteur	4	2
06.07.2026	21	création du reste de entité et continuation de la DB	4	3
07.07.2026	21	conception de messages et début de parti	4	3
08.07.2026	21	fin création de parti et début des message	3	4
09.07.2026	21	continuation des messages	3	5
10.07.2026	21	continuation sur les acteurs	4	7
13.07.2026	22	fin des acteur et début du service de simulation	3	5
14.07.2026	22	continuation sur le service de simulation	3	5
15.07.2026	22	fin de la simulation	4	7
16.07.2026	22	gestion de la reconnexion et rapport	4	7
17.07.2026	22	écriture du rapport	5	6
20.07.2026	23	écriture du rapport	4	10
21.07.2026	23	travail frontend et écriture du rapport	3	8
22.07.2026	23	travail frontend et écriture du rapport	4	8
23.07.2026	23	travail frontend et écriture du rapport	4	11
24.07.2026	23	finalisation	10	0

Tableau 1. – Journal de travail

Bibliographie

- [1] Play Framework, « Play Framework Documentation 3.0.x ». [En ligne]. Disponible sur: <https://www.playframework.com/documentation/3.0.x/Home>
- [2] Play Framework, « Play Framework Scala Templates ». [En ligne]. Disponible sur: <https://www.playframework.com/documentation/3.0.x/ScalaTemplates>
- [3] Wikipédia, « WebGL Wikipedia ». [En ligne]. Disponible sur: <https://fr.wikipedia.org/wiki/WebGL>
- [4] PixiJS Team, « PixiJS Official Website ». [En ligne]. Disponible sur: <https://pixijs.com/>
- [5] Phaser Studio, « Phaser Official Website ». [En ligne]. Disponible sur: <https://phaser.io/>
- [6] Reddit Community, « Is It Worth Developing a Game Made Only in HTML/JS? ». [En ligne]. Disponible sur: https://www.reddit.com/r/gamedev/comments/vdp8h0/is_it_worth_developing_a_game_made_only_in/?tl=fr
- [7] Shirajuki, « JS Game Rendering Benchmark ». [En ligne]. Disponible sur: <https://github.com/Shirajuki/js-game-rendering-benchmark>
- [8] slaylines, « Canvas Engines Comparison ». [En ligne]. Disponible sur: <https://github.com/slaylines/canvas-engines-comparison>
- [9] Aaron Parecki, « Oauth2 ». [En ligne]. Disponible sur: <https://oauth.net/2/>
- [10] Ably, « What is Long Polling? ». [En ligne]. Disponible sur: <https://ably.com/topic/long-polling>
- [11] MDN Web Docs, « Using Server-Sent Events ». [En ligne]. Disponible sur: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events
- [12] Play Framework, « Play Framework Java WebSockets ». [En ligne]. Disponible sur: <https://www.playframework.com/documentation/3.0.x/JavaWebSockets>
- [13] Wikipédia, « Modèle d'acteur ». [En ligne]. Disponible sur: https://fr.wikipedia.org/wiki/Mod%C3%A8le_d%27acteur
- [14] Lukas Eder, « Using jOOQ with Play Framework ». [En ligne]. Disponible sur: <https://blog.jooq.org/using-jooq-with-play-framework/>
- [15] Play Framework, « Play Framework Database Documentation ». [En ligne]. Disponible sur: <https://www.playframework.com/documentation/3.0.x/Databases>

- [16] Riot Games, « Thinking Inside Container ». [En ligne]. Disponible sur: <https://www.riotgames.com/en/news/thinking-inside-container>

Table des figures

Fig. 1	Plannification	15
Fig. 2	Schéma architecture logiciel	23
Fig. 3	UML du HomeController	29
Fig. 4	UML de l'Unité	30
Fig. 5	UML des Effets	31
Fig. 6	UML de l'utilisateur	32
Fig. 7	UML de la partie	33
Fig. 8	UML de l'équipe	34
Fig. 9	Schéma replay sans Json	36
Fig. 10	Schéma replay avec Json	37
Fig. 11	Schéma de séquence user input	39
Fig. 12	UML team/Event	41
Fig. 13	UML UnitEvent	42
Fig. 14	UML changingShopEvent	43
Fig. 15	Schéma de séquence de l'authentification	45
Fig. 16	UML des acteurs	46
Fig. 17	Schéma de séquence des pannes	48
Fig. 18	schéma d'activité ECS	50
Fig. 19	schéma explicatif des rprésentation cubique et classique	62

Annexes

I Outils utilisés

pour ce projet j'ai utilisé les LLms : claude et chatgpt Durant ce projet voici mon utilisation des LLMs:

- Correction de l'orthographe
- Quelques propositions de reformulation de phrase
- Explication de sujet théorique
- Aide à la recherche de source sur les Technologies utilisées dans mes inspirations (TFT, hearthstone)
- Aide à l'implémentation du pathFinding
- Aide à l'implémentation de tests
- Aide à l'implémentation du frontend